Predicting Student Romantic Relationship Status

Weiyang Sheng

Literature review and research motivation

Adolescent romantic relationships are not simply a private or trivial part of teenage life. Developmental research treats them as an important social context in which young people practice intimacy, emotion regulation, identity exploration, peer negotiation, and future relationship expectations. Collins (2003) argued that adolescent romantic relationships should be studied through several dimensions, including involvement, partner selection, relationship content, relationship quality, and cognitive-emotional processes. This matters for the present project because the outcome variable, `romantic`, is not just a yes/no label; it is a developmental indicator of whether a student is already participating in a meaningful social and emotional domain.

A broader review by Collins, Welsh, and Furman (2009) also emphasizes that romantic relationships from early adolescence into the early twenties are multifaceted and developmentally significant. Their review suggests that adolescent romance is connected to peer relationships, family context, emotional experiences, and age-related changes. This project therefore uses a wide set of predictors rather than relying on only one narrow explanation. In the dataset, possible predictors include sex, age, absences, study time, free time, going out, alcohol use, internet access, extracurricular activities, family background, and school-related variables. The goal is not to claim that these variables cause romantic relationships, but to test which variables help predict romantic involvement in a real student sample.

Research also shows that adolescent romantic involvement is common enough to be studied empirically. Carver, Joyner, and Udry (2003), using Add Health data, describe adolescent romantic relationships as a central concern for both adolescents and researchers studying the transition to adulthood. Meier and Allen (2009) further show continuity from adolescent romantic relationships into young adulthood, which means that early relationship experiences may be part of a longer developmental pathway rather than isolated events. For a school or youth-development context, this makes the prediction question practically meaningful: if some students are more likely to be in relationships, counselors and educators may better

understand which social, behavioral, and emotional contexts are most connected to romantic involvement.

At the same time, romantic relationships can be linked to both positive and negative outcomes. A systematic review by Gómez-López, Viejo, and Ortega-Ruiz (2019) found that romantic relationships can be a source of well-being, but the literature is heterogeneous and relationship effects depend on quality, context, and measurement. Longitudinal evidence also connects romantic relationship quality and depressive symptoms during late adolescence (Joosten et al., 2022), while earlier work by Joyner and Udry (2000) used Add Health data to examine associations between adolescent romance and depression. These findings make it important to study romantic involvement carefully: being in a relationship is not automatically good or bad, but it is a meaningful social marker that may relate to mental health, behavior, and social support.

There is also a public-health and educational reason for studying adolescent relationship patterns. The CDC's Dating Matters/Healthy Relationships Toolkit focuses on teaching 11- to 14-year-olds healthy relationship skills before they start dating, and it aims to reduce risk factors such as dating violence, substance abuse, and sexual risk-taking. This does not mean that the current project is trying to identify or label individual students for intervention. Instead, it means that a prediction-focused analysis can help reveal which kinds of student characteristics are most connected to romantic involvement. In a responsible interpretation, these models should be used to understand patterns and improve support, not to stereotype students or make deterministic claims.

The present project therefore asks a focused predictive question: **Which student characteristics best predict whether a high-school student reports being in a romantic relationship?** The project uses the Kaggle **Student Alcohol Consumption** dataset, which contains demographic, social, family, school-related, behavioral, and academic variables collected from Portuguese secondary-school students. The dataset includes two course files, `student-mat.csv` for Math and `student-por.csv` for Portuguese language. Because the same student can appear in both course files, this final analysis first merges repeated student records into one student-level dataset. The modeling section then compares interpretable logistic regression, flexible machine-learning models, an ensemble layer, and probability-calibration diagnostics.

Key literature used to motivate the project. Key literature and data sources used to motivate the project include Collins (2003); Collins, Welsh, and Furman (2009); Carver, Joyner, and Udry (2003); Meier and Allen (2009); Gómez-López, Viejo, and Ortega-Ruiz (2019); Joosten et al. (2022); Joyner and Udry (2000); CDC Dating Matters; and the Kaggle Student Alcohol Consumption dataset data card.

Project setup

This chunk loads the packages used throughout the analysis and fixes the random seed so the workflow is reproducible. It provides the software foundation for the entire project.

```
# This chunk loads the packages used throughout the project.
# It also fixes the random seed so that the workflow is reproducible.
# The goal is to keep the analysis stable and transparent.
library(tidyverse)
library(tidymodels)
library(vip)
library(stacks)
library(xgboost)
library(kernlab)
library(nnet)
library(finetune)
library(broom)
library(glue)
library(ranger)

options(yardstick.event_first = FALSE)
set.seed(123)
```

This chunk does not produce a substantive analytical result by itself. Its main purpose is to prepare a stable and reproducible environment for the rest of the project.

Data import and sample construction

This chunk reads the Math and Portuguese course datasets, identifies overlapping students, and constructs a cleaner merged-student sample. Its role in the paper is to show that the final analysis is based on student-level data rather than double-counting the same student across two course files.

```
# This chunk reads the Math and Portuguese datasets.
# It identifies overlapping students and merges repeated records into one student-level row.
# The goal is to build the main analysis sample and avoid double-counting the same student.
student_por <- read_csv("student-por.csv", show_col_types = FALSE)
student_mat <- read_csv("student-mat.csv", show_col_types = FALSE)

student_mat <- student_mat %>% mutate(course = "Math")
student_por <- student_por %>% mutate(course = "Portuguese")
```

```r
id_vars <- c(
  "school", "sex", "age", "address", "famsize", "Pstatus",
  "Medu", "Fedu", "Mjob", "Fjob", "reason", "nursery", "internet"
)

dup_students <- inner_join(
  student_mat %>% dplyr::select(all_of(id_vars)) %>% distinct(),
  student_por %>% dplyr::select(all_of(id_vars)) %>% distinct(),
  by = id_vars
)

nrow(dup_students) # overlap count based on id_vars
```

```
[1] 366
```

```r
mode_value <- function(x) {
  x <- x[!is.na(x)]
  if (length(x) == 0) {
    return(NA_character_)
  }
  ux <- unique(x)
  ux[which.max(tabulate(match(x, ux)))]
}

rounded_mean_or_na <- function(x) {
  if (all(is.na(x))) {
    return(NA_real_)
  }
  round(mean(as.numeric(x), na.rm = TRUE))
}

mean_or_na <- function(x) {
  if (all(is.na(x))) {
    return(NA_real_)
  }
  mean(as.numeric(x), na.rm = TRUE)
}

# Keep these variables on their original discrete scales so merging does not create awkward v
rounded_merge_vars <- c(
  "traveltime", "studytime", "failures", "famrel",
```

```
  "freetime", "goout", "Dalc", "Walc", "health"
)

# These variables can be merged using the mean.
mean_merge_vars <- c("absences", "G1", "G2", "G3")

# Merge yes/no variables using the mode.
yesno_merge_vars <- c("schoolsup", "famsup", "paid", "activities", "higher", "romantic")

# Merge other categorical variables using the mode.
categorical_merge_vars <- c("guardian")

student_all <- bind_rows(student_mat, student_por) %>%
  tidyr::unite("student_id", all_of(id_vars), sep = "__", remove = FALSE) %>%
  dplyr::group_by(student_id) %>%
  dplyr::summarise(
    dplyr::across(all_of(id_vars), ~ dplyr::first(.x)),
    course = dplyr::if_else(dplyr::n_distinct(course) > 1, "Both", dplyr::first(course)),
    dplyr::across(any_of(rounded_merge_vars), rounded_mean_or_na),
    dplyr::across(any_of(mean_merge_vars), mean_or_na),
    dplyr::across(any_of(yesno_merge_vars), ~ mode_value(as.character(.x))),
    dplyr::across(any_of(categorical_merge_vars), ~ mode_value(as.character(.x))),
    .groups = "drop"
  ) %>%
  dplyr::select(-student_id)

nrow(student_all)
```

```
[1] 662
```

```
readr::write_csv(student_all, "student_all_merged.csv")
```

Interpretation. This output shows that there is substantial overlap between the two course files, so merging repeated records is necessary before modeling. After repeated records are combined, the final analytic sample contains 662 unique students. In simple terms, the project is working at the student level rather than double-counting the same person across two subjects.

Data cleaning and feature engineering

This chunk converts raw variables into analysis-ready predictors, including parental education percentages, SES-style job groupings, age groups, and alcohol-related measures. In the final analysis, the project uses one pooled sample only, and sex is kept as an explicit predictor rather than splitting the main modeling pipeline into separate female and male branches.

```
# This chunk converts raw variables into analysis-ready features.
# It recodes variables, creates grouped and percentage-based predictors,
# and builds the train/test split for the pooled analysis only.
students <- student_all %>%
  mutate(
    romantic = factor(romantic, levels = c("no", "yes")),
    sex = factor(sex),
    school = factor(school),
    address = factor(address),
    famsize = factor(famsize),
    Pstatus = factor(Pstatus),
    Mjob = factor(Mjob),
    Fjob = factor(Fjob),
    guardian = factor(guardian),
    course = factor(course, levels = c("Math", "Portuguese", "Both")),
    schoolsup = factor(schoolsup, levels = c("no", "yes")),
    famsup = factor(famsup, levels = c("no", "yes")),
    paid = factor(paid, levels = c("no", "yes")),
    activities = factor(activities, levels = c("no", "yes")),
    higher = factor(higher, levels = c("no", "yes")),
    internet = factor(internet, levels = c("no", "yes")),
    Medu = as.numeric(Medu),
    Fedu = as.numeric(Fedu),
    age = as.numeric(age),
    traveltime = as.numeric(traveltime),
    studytime = as.numeric(studytime),
    failures = as.numeric(failures),
    famrel = as.numeric(famrel),
    freetime = as.numeric(freetime),
    goout = as.numeric(goout),
    Dalc = as.numeric(Dalc),
    Walc = as.numeric(Walc),
    health = as.numeric(health),
    absences = as.numeric(absences),
    G1 = as.numeric(G1),
    G2 = as.numeric(G2),
```

```
    G3 = as.numeric(G3),
    Medu_pct = 100 * Medu / 4,
    Fedu_pct = 100 * Fedu / 4,
    edu_gap_pct = abs(Medu_pct - Fedu_pct),
    Mjob_ses = case_when(
      Mjob %in% c("teacher", "health") ~ "higher",
      Mjob %in% c("services") ~ "middle",
      TRUE ~ "lower"
    ),
    Mjob_ses = factor(Mjob_ses, levels = c("lower", "middle", "higher")),
    Fjob_ses = case_when(
      Fjob %in% c("teacher", "health") ~ "higher",
      Fjob %in% c("services") ~ "middle",
      TRUE ~ "lower"
    ),
    Fjob_ses = factor(Fjob_ses, levels = c("lower", "middle", "higher")),
    age_group = case_when(
      age <= 16 ~ "15-16",
      age == 17 ~ "17",
      age == 18 ~ "18",
      age >= 19 ~ "19+"
    ),
    age_group = factor(age_group, levels = c("15-16", "17", "18", "19+")),
    alc_mean = (Dalc + Walc) / 2,
    alc_risk = factor(if_else(Walc >= 4 | Dalc >= 3, "higher", "lower"),
      levels = c("lower", "higher")
    ),
    freetime_pct = 100 * (freetime - 1) / 4,
    goout_pct = 100 * (goout - 1) / 4,
    studytime_pct = 100 * (studytime - 1) / 3,
    alc_mean_pct = 100 * (alc_mean - 1) / 4
  )

set.seed(123)
splits <- initial_split(students, prop = 0.8, strata = romantic)
train_data <- training(splits)
test_data <- testing(splits)

train_data_mod <- train_data
test_data_mod <- test_data

# Build lightweight pooled datasets without sex for the later ablation check.
```

```r
train_data_nosex <- train_data_mod %>% dplyr::select(-sex)
test_data_nosex <- test_data_mod %>% dplyr::select(-sex)

prop.table(table(train_data$romantic))
```

```
       no       yes
0.6257089 0.3742911
```

```r
prop.table(table(test_data$romantic))
```

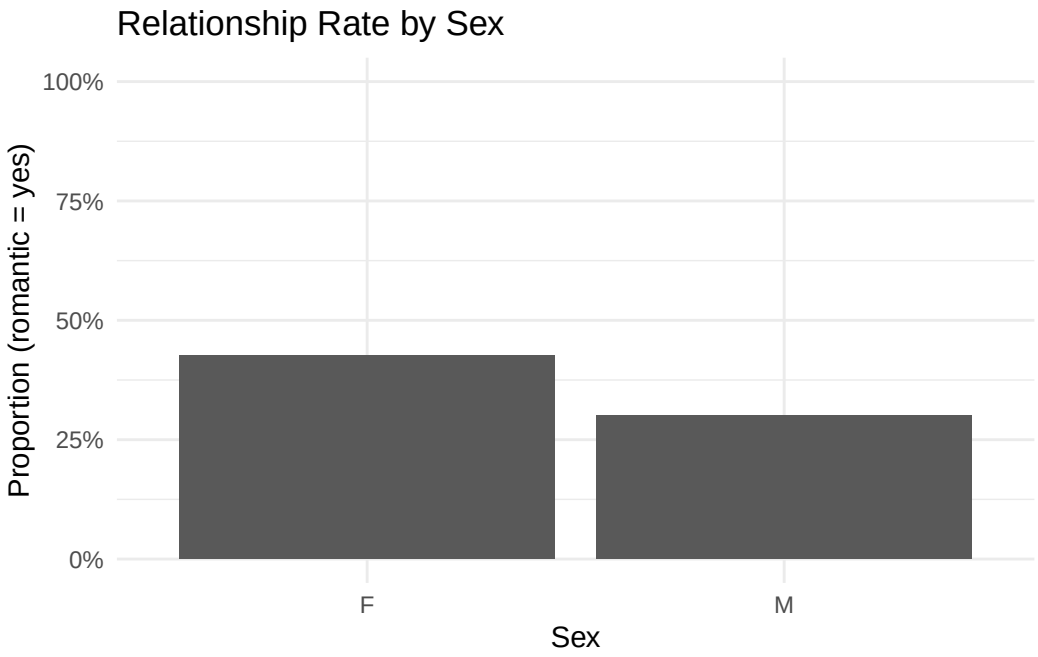
```
       no       yes
0.6240602 0.3759398
```

Interpretation. These outputs show that the train and test sets have almost the same class balance. About 62.5% of students are in the `no` group and about 37.5% are in the `yes` group in both splits. That means the stratified split worked well and the held-out test set remains comparable to the training data.

Plot analysis: relationship rate by sex

This chunk gives the overall relationship rate for female and male students. Its role is to establish the baseline sex difference before moving into pooled predictive modeling with sex retained as a predictor.

```r
# This chunk provides the overall relationship rate for female and male students.
# It is meant to establish the baseline sex difference before moving into
# pooled modeling where sex is kept as a predictor.
students %>%
  group_by(sex) %>%
  summarise(prop_yes = mean(romantic == "yes"), n = n(), .groups = "drop") %>%
  ggplot(aes(x = sex, y = prop_yes)) +
  geom_col() +
  scale_y_continuous(labels = scales::percent_format(), limits = c(0, 1)) +
  labs(
    title = "Relationship Rate by Sex",
    x = "Sex",
    y = "Proportion (romantic = yes)"
  ) +
  theme_minimal()
```



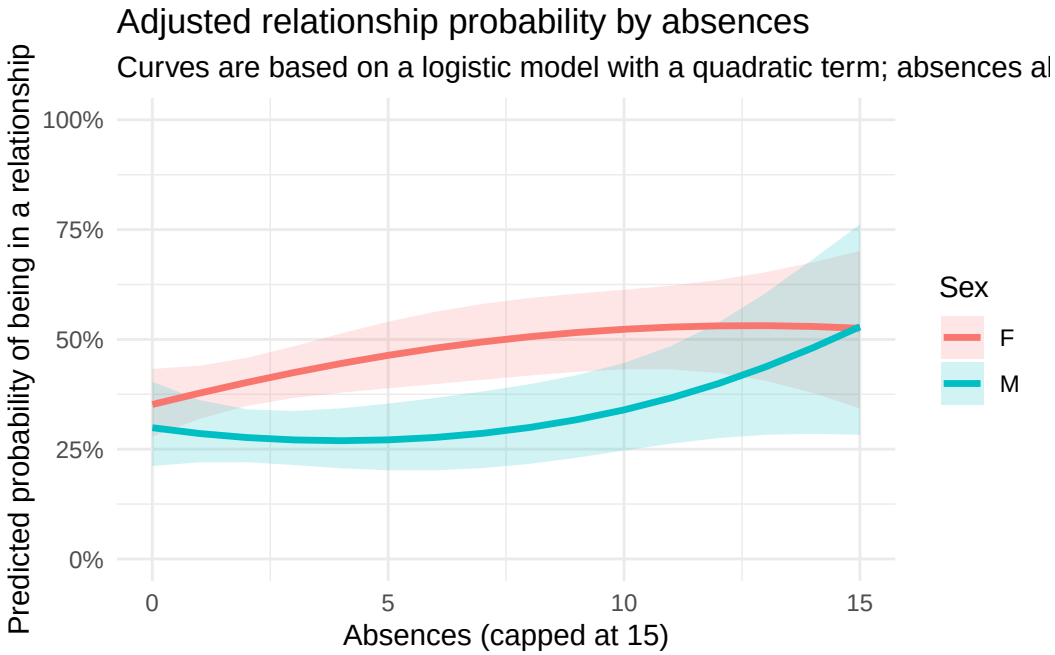
Interpretation. In this bar plot, the **x-axis** separates students by sex (`F` and `M`), and the **y-axis** shows the proportion of students who answered `yes` to the romantic-relationship question. Each bar therefore represents the observed relationship rate within one sex group. The female bar is visibly higher than the male bar, meaning that female students in this dataset report romantic relationships more often than male students. This result has two practical implications. First, sex should not be discarded as a background variable because it contains predictive information. Second, the project should be careful not to treat all students as socially identical; relationship involvement appears to differ by demographic and social context.

Plot analysis: adjusted absences effect

This chunk uses a logistic model to estimate the adjusted relationship between absences and the probability of being in a relationship. Its role is to move beyond raw grouped percentages and show a smoother, more interpretable pattern for one of the clearest predictors in the current results.

```
# This chunk fits a logistic model to estimate the adjusted relationship between absences
# and the probability of being in a relationship. It uses a quadratic term and plots
# sex-specific predicted probabilities with confidence intervals.
students_abs <- students %>%
  mutate(abs_cap = pmin(absences, 15)) # Cap very large absence values at 15 so the far tail
```

```r
abs_vis_fit <- glm(
  I(romantic == "yes") ~ poly(abs_cap, 2, raw = TRUE) * sex,
  family = binomial(),
  data = students_abs
)

abs_grid <- expand.grid(
  abs_cap = 0:15,
  sex = levels(students_abs$sex)
)

abs_pred <- predict(abs_vis_fit, newdata = abs_grid, type = "link", se.fit = TRUE)

abs_plot_df <- abs_grid %>%
  mutate(
    fit_link = abs_pred$fit,
    se_link = abs_pred$se.fit,
    prob = plogis(fit_link),
    lower = plogis(fit_link - 1.96 * se_link),
    upper = plogis(fit_link + 1.96 * se_link)
  )

ggplot(abs_plot_df, aes(x = abs_cap, y = prob, color = sex, fill = sex)) +
  geom_ribbon(aes(ymin = lower, ymax = upper), alpha = 0.18, color = NA) +
  geom_line(linewidth = 1.2) +
  scale_y_continuous(labels = scales::percent_format(), limits = c(0, 1)) +
  labs(
    title = "Adjusted relationship probability by absences",
    subtitle = "Curves are based on a logistic model with a quadratic term; absences above 15
    x = "Absences (capped at 15)",
    y = "Predicted probability of being in a relationship",
    color = "Sex",
    fill = "Sex"
  ) +
  theme_minimal()
```



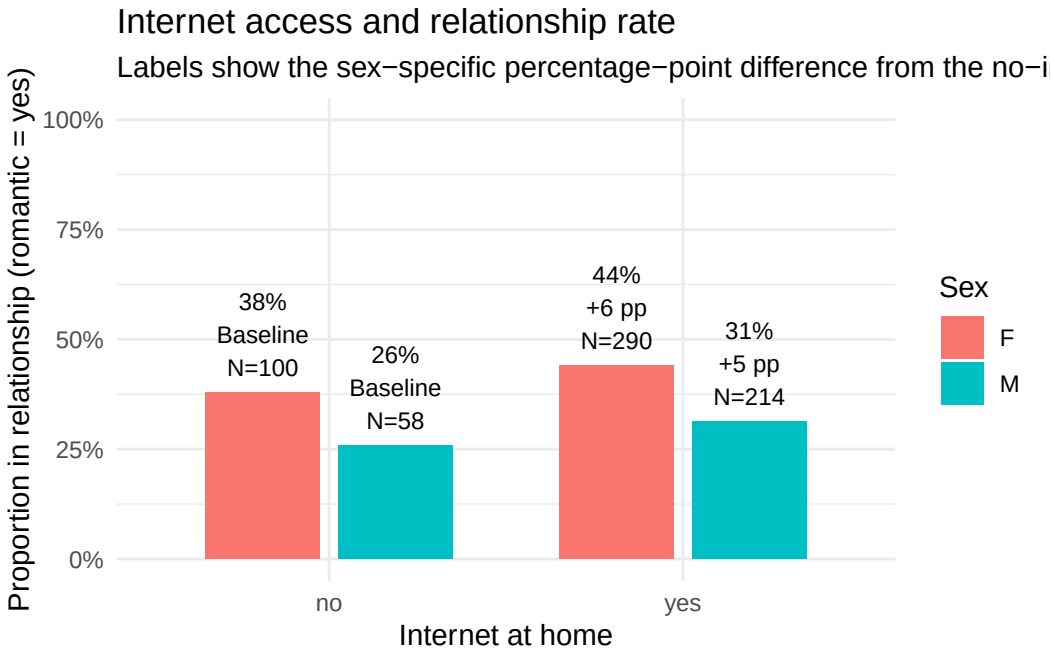
Interpretation. In this adjusted plot, the **x-axis** is the number of school absences after capping very high values at 15, and the **y-axis** is the model-predicted probability that a student is in a romantic relationship. The two colored curves separate female and male students, while the shaded ribbons show uncertainty around the estimated curves. The upward movement at higher absence levels suggests that students with more absences tend to have higher predicted relationship probability. This does not prove that dating causes absence or that absence causes dating, but it suggests that romantic involvement is connected to broader behavioral routines and school engagement. For educators, this means absence patterns may be useful contextual information when understanding students' social lives.

Plot analysis: internet access

This chunk visualizes internet access as a simple and interpretable binary predictor, with labels showing the sex-specific percentage-point difference from the no-internet group. Its role is to highlight a direct opportunity/resource variable that is easier to interpret than the weaker parent-education effects.

```
# This chunk compares relationship rates between students with and without internet at home.
# It also labels the sex-specific percentage-point difference relative to the no-internet gr
# making the result easy to interpret directly.
internet_df <- students %>%
  group_by(sex, internet) %>%
```

```r
  summarise(
    prop_yes = mean(romantic == "yes"),
    n = n(),
    .groups = "drop"
  ) %>%
  group_by(sex) %>%
  mutate(
    base_no = prop_yes[internet == "no"],
    diff_pp = 100 * (prop_yes - base_no)
  ) %>%
  ungroup()

ggplot(internet_df, aes(x = internet, y = prop_yes, fill = sex)) +
  geom_col(position = position_dodge(width = 0.75), width = 0.65) +
  geom_text(
    aes(
      label = if_else(
        internet == "yes",
        paste0(scales::percent(prop_yes, accuracy = 1), "\n", sprintf("%+.0f pp", diff_pp), "
        paste0(scales::percent(prop_yes, accuracy = 1), "\nBaseline\nN=", n)
      )
    ),
    position = position_dodge(width = 0.75),
    vjust = -0.2,
    size = 3
  ) +
  scale_y_continuous(labels = scales::percent_format(), limits = c(0, 1)) +
  labs(
    title = "Internet access and relationship rate",
    subtitle = "Labels show the sex-specific percentage-point difference from the no-interne
    x = "Internet at home",
    y = "Proportion in relationship (romantic = yes)",
    fill = "Sex"
  ) +
  theme_minimal()
```



Interpretation. In this plot, the **x-axis** shows whether the student has internet access at home, and the **y-axis** shows the observed proportion of students in a romantic relationship. The bars are grouped by sex, and the labels inside the bars show the percentage, the difference from the no-internet baseline within each sex, and the group sample size. Students with internet access show a modestly higher relationship rate in both female and male groups. This may reflect opportunity and communication access: online connection can make social interaction, planning, and peer contact easier. The effect is not huge, but it is consistent and easy to interpret, which is why internet access is retained as a relevant predictor.

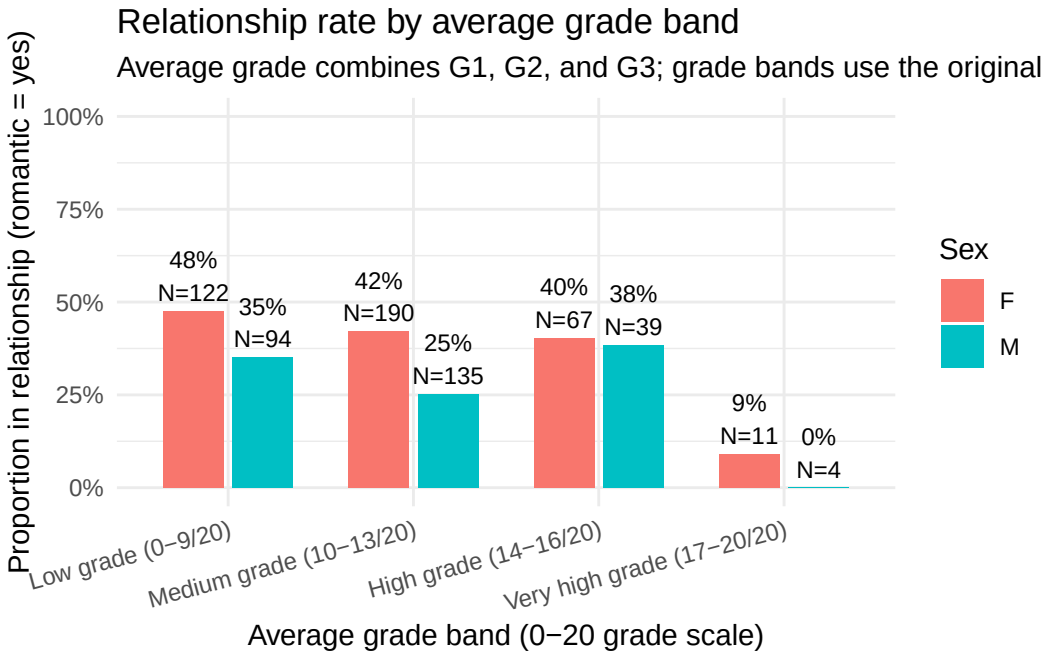
Plot analysis: academic grades and relationship rate

This chunk creates a descriptive grade-based plot. The Kaggle Student Alcohol Consumption dataset records course grades on a **0–20 academic scale**, not an age scale. Here, `G1`, `G2`, and `G3` are combined into one average grade score, and students are grouped into grade bands. This plot was added because the Random Forest variable-importance result showed that `G1`, `G2`, and `G3` were among the strongest predictors. The goal is to translate that model signal into a more intuitive descriptive picture.

```
# This chunk creates a descriptive plot for academic grades.
# G1, G2, and G3 were highly ranked in the Random Forest importance plot.
# They are combined into one average grade score on the original 0-20 scale.
# The plot then compares romantic-relationship rates across grade bands.
```

```r
grade_vars <- c("G1", "G2", "G3")

# Check that the grade variables are available in the cleaned student dataset.
missing_grade_vars <- setdiff(grade_vars, names(students))

if (length(missing_grade_vars) > 0) {
  stop(
    paste(
      "Missing grade variables:",
      paste(missing_grade_vars, collapse = ", ")
    )
  )
}

students_grade <- students %>%
  mutate(
    grade_mean = rowMeans(
      dplyr::select(., dplyr::all_of(grade_vars)),
      na.rm = TRUE
    ),
    grade_band = case_when(
      grade_mean < 10 ~ "Low grade (0-9/20)",
      grade_mean < 14 ~ "Medium grade (10-13/20)",
      grade_mean < 17 ~ "High grade (14-16/20)",
      grade_mean <= 20 ~ "Very high grade (17-20/20)",
      TRUE ~ NA_character_
    ),
    grade_band = factor(
      grade_band,
      levels = c(
        "Low grade (0-9/20)",
        "Medium grade (10-13/20)",
        "High grade (14-16/20)",
        "Very high grade (17-20/20)"
      )
    )
  )

grade_summary <- students_grade %>%
  group_by(sex, grade_band) %>%
  summarise(
    n = n(),
```

```
    yes_n = sum(romantic == "yes"),
    prop_yes = mean(romantic == "yes"),
    .groups = "drop"
  ) %>%
  tidyr::complete(
    sex,
    grade_band,
    fill = list(n = 0, yes_n = 0, prop_yes = NA_real_)
  ) %>%
  mutate(
    label = if_else(
      n > 0,
      paste0(scales::percent(prop_yes, accuracy = 1), "\nN=", n),
      "N=0"
    )
  )

ggplot(grade_summary, aes(x = grade_band, y = prop_yes, fill = sex)) +
  geom_col(position = position_dodge(width = 0.75), width = 0.65, na.rm = TRUE) +
  geom_text(
    aes(label = label),
    position = position_dodge(width = 0.75),
    vjust = -0.2,
    size = 3
  ) +
  scale_y_continuous(labels = scales::percent_format(), limits = c(0, 1)) +
  labs(
    title = "Relationship rate by average grade band",
    subtitle = "Average grade combines G1, G2, and G3; grade bands use the original 0-20 scal
    x = "Average grade band (0-20 grade scale)",
    y = "Proportion in relationship (romantic = yes)",
    fill = "Sex"
  ) +
  theme_minimal() +
  theme(axis.text.x = element_text(angle = 15, hjust = 1))
```



Interpretation. In this plot, the **x-axis** groups students by average academic grade across `G1`, `G2`, and `G3`; these are grades on a **0–20 scale**, so "0–9/20" means a low grade range, not student age. The **y-axis** shows the proportion of students in each grade band who reported `romantic = yes`. The bars are separated by sex, and each label shows both the observed percentage and the group sample size. The pattern suggests that lower and medium grade bands have higher relationship rates, while the very-high-grade group has a much lower observed rate. However, the very-high-grade group is small, especially for males, so that part should be interpreted cautiously. Substantively, this figure helps explain why grade variables were important in the Random Forest model: academic performance appears to be connected to students' romantic relationship status, but this association is descriptive rather than causal.

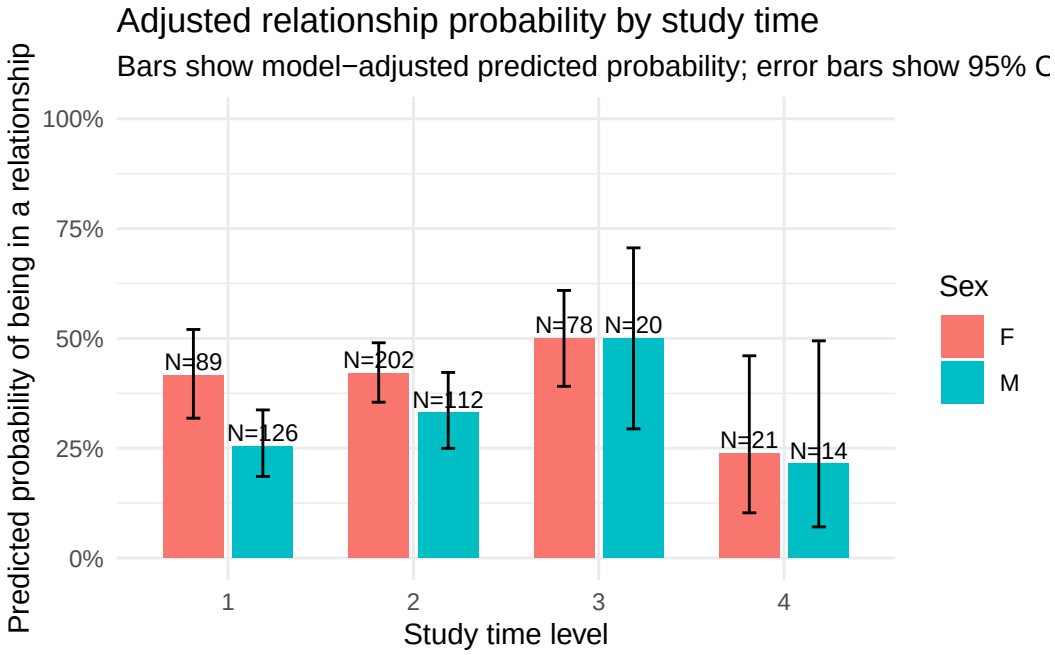
Plot analysis: adjusted study time effect

This chunk shows the model-adjusted probability of being in a relationship across study-time levels, separately by sex. Its role is to present study time as a level-based predictor rather than forcing it into a misleading straight-line interpretation.

```
# This chunk estimates the model-adjusted probability of being in a relationship
# across study-time levels, separately by sex. It treats study time as a level-based
# predictor rather than forcing it into a simple straight-line effect.
study_vis_fit <- glm(
  I(romantic == "yes") ~ factor(studytime) * sex,
```

```r
  family = binomial(),
  data = students
)

study_grid <- expand.grid(
  studytime = sort(unique(students$studytime)),
  sex = levels(students$sex)
)

study_pred <- predict(study_vis_fit, newdata = study_grid, type = "link", se.fit = TRUE)

study_plot_df <- study_grid %>%
  mutate(
    fit_link = study_pred$fit,
    se_link = study_pred$se.fit,
    prob = plogis(fit_link),
    lower = plogis(fit_link - 1.96 * se_link),
    upper = plogis(fit_link + 1.96 * se_link)
  ) %>%
  left_join(
    students %>%
      group_by(sex, studytime) %>%
      summarise(n = n(), .groups = "drop"),
    by = c("sex", "studytime")
  )

ggplot(study_plot_df, aes(x = factor(studytime), y = prob, fill = sex)) +
  geom_col(position = position_dodge(width = 0.75), width = 0.65) +
  geom_errorbar(
    aes(ymin = lower, ymax = upper),
    position = position_dodge(width = 0.75),
    width = 0.15
  ) +
  geom_text(
    aes(label = paste0("N=", n)),
    position = position_dodge(width = 0.75),
    vjust = -0.3,
    size = 3
  ) +
  scale_y_continuous(labels = scales::percent_format(), limits = c(0, 1)) +
  labs(
    title = "Adjusted relationship probability by study time",
```

```
    subtitle = "Bars show model-adjusted predicted probability; error bars show 95% CI",
    x = "Study time level",
    y = "Predicted probability of being in a relationship",
    fill = "Sex"
  ) +
  theme_minimal()
```



Interpretation. In this plot, the **x-axis** shows the four study-time levels from the survey, and the **y-axis** shows the model-adjusted predicted probability of being in a romantic relationship. The bars are separated by sex, the error bars show uncertainty, and the `N=` labels show how many students are represented at each level. The pattern is not a simple straight line. Moderate study-time levels often show higher relationship probability, while the highest study-time group is lower and also has fewer students. This suggests a realistic social tradeoff: some study commitment may coexist with active social life, but very intensive study schedules may limit social opportunity or reflect students with different priorities.

Modeling approach: preprocessing recipe

This chunk defines the preprocessing pipeline used before machine-learning models are fitted. Its role is to make the predictive comparison fair by standardizing numeric variables, collapsing rare categories, removing redundant predictors, and creating dummy-coded inputs where needed.

```r
# This chunk defines the preprocessing pipeline for the pooled model.
# It removes redundant variables, collapses rare categories,
# standardizes numeric predictors, and creates dummy-coded features for modeling.
rec_all <- recipe(romantic ~ ., data = train_data_mod) %>%
  step_rm(
    course, age, Dalc, Walc, Mjob, Fjob, Medu, Fedu,
    freetime_pct, goout_pct, studytime_pct, alc_mean_pct
  ) %>%
  step_other(all_nominal_predictors(), threshold = 0.03, other = "other_collapsed") %>%
  step_nzv(all_predictors()) %>%
  step_normalize(all_numeric_predictors()) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_zv(all_predictors())

# This second recipe is identical except that sex is removed from the predictor set.
# It is used only for a concise pooled ablation check on whether keeping sex helps.
rec_all_nosex <- recipe(romantic ~ ., data = train_data_nosex) %>%
  step_rm(
    course, age, Dalc, Walc, Mjob, Fjob, Medu, Fedu,
    freetime_pct, goout_pct, studytime_pct, alc_mean_pct
  ) %>%
  step_other(all_nominal_predictors(), threshold = 0.03, other = "other_collapsed") %>%
  step_nzv(all_predictors()) %>%
  step_normalize(all_numeric_predictors()) %>%
  step_dummy(all_nominal_predictors()) %>%
  step_zv(all_predictors())
```

This chunk is a preprocessing step rather than a substantive result. Its role is to make the later model comparison fair by giving all pooled models a consistent input structure.

Baseline model: logistic regression

This chunk fits logistic regression models for the pooled sample. Its role is to provide an interpretable statistical baseline before comparing more flexible nonlinear models.

```r
# This chunk fits the baseline logistic-regression model for the pooled sample.
# It provides an interpretable benchmark before moving to more flexible machine-learning mode
logit_spec <- logistic_reg() %>% set_engine("glm")

logit_wf <- workflow() %>%
  add_recipe(rec_all) %>%
```

```
  add_model(logit_spec)
logit_fit <- fit(logit_wf, data = train_data_mod)

# This second pooled logistic model removes sex and is used for a direct pooled ablation che
logit_wf_nosex <- workflow() %>%
  add_recipe(rec_all_nosex) %>%
  add_model(logit_spec)
logit_fit_nosex <- fit(logit_wf_nosex, data = train_data_nosex)

logit_fit %>%
  extract_fit_parsnip() %>%
  tidy() %>%
  arrange(p.value)
```

```
# A tibble: 41 x 5
   term            estimate std.error statistic  p.value
   <chr>              <dbl>     <dbl>     <dbl>    <dbl>
 1 sex_M             -0.865     0.236     -3.67 0.000241
 2 activities_yes     0.471     0.207      2.27 0.0232
 3 internet_yes       0.522     0.249      2.10 0.0361
 4 Mjob_ses_middle   -0.583     0.279     -2.09 0.0365
 5 absences           0.227     0.112      2.03 0.0428
 6 schoolsup_yes     -0.754     0.387     -1.95 0.0511
 7 age_group_X18      0.455     0.268      1.70 0.0892
 8 G1                 0.336     0.206      1.63 0.104
 9 guardian_other     0.715     0.520      1.37 0.169
10 age_group_X17      0.337     0.246      1.37 0.171
# i 31 more rows
```

Interpretation. This coefficient table reports the pooled logistic baseline on the log-odds scale. The clearest signals near the top are usually sex, absences, activity-related variables, and access-related variables rather than the weaker family-background terms. In practical terms, the pooled baseline already points more strongly toward sex and behavior-related variables than toward parental background alone.

Tuned machine-learning models and ensemble construction

This chunk tunes multiple predictive models, including Random Forest, Elastic Net, SVM, MLP, XGBoost, a decision tree, and a stacking ensemble. A distance-based k-nearest-neighbor model was explored during model development, but it is not retained in the main result table because it was less stable in the pooled Accuracy-first evaluation. The main purpose of this

chunk is to test whether more flexible models improve prediction beyond the interpretable logistic baseline while keeping the analysis at the pooled student level.

```
# This chunk tunes several machine-learning models, including Random Forest,
# Elastic Net, SVM, MLP, XGBoost, a decision tree, and a stacking ensemble.
# A k-nearest-neighbor model was explored during development but is excluded
# from the final main result table because it was unstable in the pooled test evaluation.
# The goal is to compare flexible pooled predictive models against the logistic baseline.
metric_cls <- metric_set(accuracy, roc_auc)

rf_spec_tune <- rand_forest(
  mode  = "classification",
  trees = tune(),
  mtry  = tune(),
  min_n = tune()
) %>%
  set_engine("ranger", importance = "impurity")

enet_spec <- logistic_reg(penalty = tune(), mixture = tune()) %>%
  set_engine("glmnet", maxit = 200000) %>%
  set_mode("classification")

svm_spec <- svm_rbf(cost = tune(), rbf_sigma = tune()) %>%
  set_engine("kernlab") %>%
  set_mode("classification")


mlp_spec <- mlp(hidden_units = tune(), penalty = tune(), epochs = tune()) %>%
  set_engine("nnet") %>%
  set_mode("classification")

xgb_spec <- boost_tree(
  trees       = tune(),
  tree_depth  = tune(),
  learn_rate  = tune(),
  mtry        = tune(),
  min_n       = tune(),
  sample_size = tune()
) %>%
  set_engine("xgboost", verbosity = 0) %>%
  set_mode("classification")

tree_spec_tune <- decision_tree(
```

```
  cost_complexity = tune(),
  tree_depth      = tune(),
  min_n           = tune()
) %>%
  set_engine("rpart") %>%
  set_mode("classification")

get_n_pred <- function(train_df, rec_obj) {
  rec_obj %>%
    prep(training = train_df, retain = TRUE) %>%
    juice() %>%
    dplyr::select(-romantic) %>%
    ncol()
}

best_mean_metric <- function(res_obj, metric_name) {
  out <- tryCatch(
    show_best(res_obj, metric = metric_name, n = 1) %>% dplyr::pull(mean),
    error = function(e) NA_real_
  )
  out[1]
}

fit_one_pooled_ensemble <- function(train_df, rec_obj, v = 10,
                                    acc_cut = 0.72,
                                    bayes_iter = 20) {
  set.seed(123)
  folds <- vfold_cv(train_df, v = v, strata = romantic)

  ctrl_grid_stack <- tune::control_grid(
    save_pred = TRUE,
    save_workflow = TRUE,
    event_level = "second",
    parallel_over = "resamples"
  )

  ctrl_bayes_stack <- tune::control_bayes(
    save_pred = TRUE,
    save_workflow = TRUE,
    event_level = "second",
    no_improve = 8,
    verbose = FALSE,
```

```
  parallel_over = "resamples"
)

ctrl_race_stack <- finetune::control_race(
  save_pred = TRUE,
  save_workflow = TRUE,
  event_level = "second",
  burn_in = 3,
  num_ties = 10,
  verbose_elim = FALSE,
  parallel_over = "resamples"
)

rf_wf <- workflow() %>%
  add_recipe(rec_obj) %>%
  add_model(rf_spec_tune)
enet_wf <- workflow() %>%
  add_recipe(rec_obj) %>%
  add_model(enet_spec)
svm_wf <- workflow() %>%
  add_recipe(rec_obj) %>%
  add_model(svm_spec)
mlp_wf <- workflow() %>%
  add_recipe(rec_obj) %>%
  add_model(mlp_spec)
xgb_wf <- workflow() %>%
  add_recipe(rec_obj) %>%
  add_model(xgb_spec)
tree_wf <- workflow() %>%
  add_recipe(rec_obj) %>%
  add_model(tree_spec_tune)

n_pred <- get_n_pred(train_df, rec_obj)

rf_params <- hardhat::extract_parameter_set_dials(rf_spec_tune) %>%
  update(
    trees = dials::trees(c(500, 2000)),
    mtry  = dials::mtry(c(2, n_pred)),
    min_n = dials::min_n(c(2, 30))
  )
rf_grid <- dials::grid_space_filling(rf_params, size = 25)
```

```r
tree_params <- hardhat::extract_parameter_set_dials(tree_spec_tune) %>%
  update(
    tree_depth      = dials::tree_depth(c(1, 12)),
    min_n           = dials::min_n(c(2, 40)),
    cost_complexity = dials::cost_complexity(c(-5, -1))
  )
tree_grid <- dials::grid_space_filling(tree_params, size = 25)

enet_params <- hardhat::extract_parameter_set_dials(enet_spec)
enet_grid <- dials::grid_space_filling(enet_params, size = 12)

svm_params <- hardhat::extract_parameter_set_dials(svm_spec)
svm_grid <- dials::grid_space_filling(svm_params, size = 15)

mlp_params <- hardhat::extract_parameter_set_dials(mlp_spec)
mlp_grid <- dials::grid_space_filling(mlp_params, size = 15)

xgb_params <- hardhat::extract_parameter_set_dials(xgb_spec) %>%
  update(
    mtry        = dials::mtry(c(2, n_pred)),
    trees       = dials::trees(c(300, 2000)),
    tree_depth  = dials::tree_depth(c(2, 10)),
    min_n       = dials::min_n(c(2, 30)),
    learn_rate  = dials::learn_rate(c(-4, -1)),
    sample_size = dials::sample_prop(c(0.5, 1.0))
  )
xgb_grid <- dials::grid_space_filling(xgb_params, size = 15)

bayes_refine <- function(wf_obj, param_info, init_grid, resamples, iter = 20) {
  set.seed(123)
  init_res <- tune_grid(
    wf_obj,
    resamples = resamples,
    grid = init_grid,
    param_info = param_info,
    metrics = metric_cls,
    control = ctrl_grid_stack
  )

  set.seed(123)
  tune_bayes(
    wf_obj,
```

```
    resamples = resamples,
    initial = init_res,
    iter = iter,
    param_info = param_info,
    metrics = metric_cls,
    control = ctrl_bayes_stack
  )
}

set.seed(123)
rf_res <- finetune::tune_race_anova(
  rf_wf,
  resamples = folds,
  grid = rf_grid,
  metrics = metric_cls,
  control = ctrl_race_stack
)


set.seed(123)
tree_res <- finetune::tune_race_anova(
  tree_wf,
  resamples = folds,
  grid = tree_grid,
  metrics = metric_cls,
  control = ctrl_race_stack
)

enet_res <- bayes_refine(enet_wf, enet_params, enet_grid, folds, iter = bayes_iter)
svm_res <- bayes_refine(svm_wf, svm_params, svm_grid, folds, iter = bayes_iter)
mlp_res <- bayes_refine(mlp_wf, mlp_params, mlp_grid, folds, iter = bayes_iter)
xgb_res <- bayes_refine(xgb_wf, xgb_params, xgb_grid, folds, iter = bayes_iter)

family_tbl <- tibble(
  family = c("rf", "xgb", "mlp", "svm", "tree", "enet"),
  auc = c(
    best_mean_metric(rf_res, "roc_auc"),
    best_mean_metric(xgb_res, "roc_auc"),
    best_mean_metric(mlp_res, "roc_auc"),
    best_mean_metric(svm_res, "roc_auc"),
    best_mean_metric(tree_res, "roc_auc"),
    best_mean_metric(enet_res, "roc_auc")
```

```
  ),
  acc = c(
    best_mean_metric(rf_res, "accuracy"),
    best_mean_metric(xgb_res, "accuracy"),
    best_mean_metric(mlp_res, "accuracy"),
    best_mean_metric(svm_res, "accuracy"),
    best_mean_metric(tree_res, "accuracy"),
    best_mean_metric(enet_res, "accuracy")
  )
) %>%
  arrange(desc(acc), desc(auc))

families_keep <- family_tbl %>%
  filter(acc >= acc_cut) %>%
  pull(family)

if (length(families_keep) < 3) {
  families_keep <- family_tbl %>%
    slice_head(n = 3) %>%
    pull(family)
}

stack_obj <- stacks()

if ("rf" %in% families_keep) stack_obj <- stack_obj %>% add_candidates(rf_res)
if ("xgb" %in% families_keep) stack_obj <- stack_obj %>% add_candidates(xgb_res)
if ("mlp" %in% families_keep) stack_obj <- stack_obj %>% add_candidates(mlp_res)
if ("svm" %in% families_keep) stack_obj <- stack_obj %>% add_candidates(svm_res)
if ("tree" %in% families_keep) stack_obj <- stack_obj %>% add_candidates(tree_res)
if ("enet" %in% families_keep) stack_obj <- stack_obj %>% add_candidates(enet_res)

stack_fit <- tryCatch(
  stack_obj %>%
    blend_predictions(
      penalty = 10^seq(-6, -1, length.out = 40),
      metric = metric_set(accuracy)
    ) %>%
    fit_members(),
  error = function(e) NULL
)

fit_best <- function(wf_obj, res_obj, train_df_inner) {
```

```
    best_params <- tune::select_best(res_obj, metric = "accuracy")
    wf_final <- tune::finalize_workflow(wf_obj, best_params)
    fit(wf_final, data = train_df_inner)
  }

  list(
    folds = folds,
    family_tbl = family_tbl,
    families_keep = families_keep,
    rf_best = tune::select_best(rf_res, metric = "accuracy"),
    xgb_best = tune::select_best(xgb_res, metric = "accuracy"),
    tree_best = tune::select_best(tree_res, metric = "accuracy"),
    rf_fit = fit_best(rf_wf, rf_res, train_df),
    enet_fit = fit_best(enet_wf, enet_res, train_df),
    svm_fit = fit_best(svm_wf, svm_res, train_df),
    mlp_fit = fit_best(mlp_wf, mlp_res, train_df),
    xgb_fit = fit_best(xgb_wf, xgb_res, train_df),
    tree_fit = fit_best(tree_wf, tree_res, train_df),
    stack_fit = stack_fit,
    rf_res = rf_res,
    xgb_res = xgb_res,
    tree_res = tree_res,
    svm_res = svm_res,
    enet_res = enet_res,
    mlp_res = mlp_res
  )
}

ens_all <- fit_one_pooled_ensemble(train_data_mod, rec_all, v = 10, acc_cut = 0.72, bayes_it

rf_fit <- ens_all$rf_fit
enet_fit <- ens_all$enet_fit
svm_fit <- ens_all$svm_fit
mlp_fit <- ens_all$mlp_fit
xgb_fit <- ens_all$xgb_fit
tree_fit <- ens_all$tree_fit
stack_fit_all <- ens_all$stack_fit

family_tbl_main <- ens_all$family_tbl %>%
  arrange(desc(acc), desc(auc))

family_tbl_main
```

```
# A tibble: 6 x 3
  family   auc   acc
  <chr>  <dbl> <dbl>
1 xgb    0.606 0.647
2 rf     0.594 0.633
3 svm    0.589 0.631
4 tree   0.531 0.629
5 enet   0.584 0.626
6 mlp    0.560 0.614
```

```
ens_all$families_keep
```

```
[1] "xgb" "rf"  "svm"
```

```
ens_all$rf_best
```

```
# A tibble: 1 x 4
   mtry trees min_n .config
  <int> <int> <int> <chr>
1     3  1500    24 Preprocessor1_Model02
```

```
ens_all$xgb_best
```

```
# A tibble: 1 x 7
   mtry trees min_n tree_depth learn_rate sample_size .config
  <int> <int> <int>      <int>      <dbl>       <dbl> <chr>
1    30  1805    28          2    0.00293       0.863 Iter8
```

```
ens_all$tree_best
```

```
# A tibble: 1 x 4
  cost_complexity tree_depth min_n .config
            <dbl>      <int> <int> <chr>
1       0.0000316          2    13 Preprocessor1_Model04
```

Cross-validated model-family ranking. This output summarizes the pooled cross-validated performance of the retained model families before final test-set evaluation. The table reports the family name, cross-validated AUC, and cross-validated Accuracy. This step helps identify which model families are promising before they are evaluated on the

held-out test set. A distance-based k-nearest-neighbor model was explored during model development, but it is not included in the final main table because it was less stable in the pooled Accuracy-first evaluation.

Families kept for stacking and best hyperparameters. These outputs show which model families were retained for the ensemble and what the selected tuning values look like. In practical terms, the tuning stage is favoring the more stable pooled candidates rather than extreme or highly unstable fits.

Model performance comparison

This chunk compares all candidate models on the held-out test set. Its role in the paper is to determine which pooled models perform best under the same Accuracy-first evaluation rule, while still showing the added technical layer of average-probability and stacking ensembles.

```
# This chunk evaluates all candidate models on the held-out test set.
# It selects a threshold on the training data, applies that threshold to the test data,
# and compares pooled models using Accuracy, F1, and AUC together.
best_threshold <- function(truth, prob,
                           optimize_for = c("accuracy", "f1"),
                           grid = seq(0.05, 0.95, by = 0.01)) {
  optimize_for <- match.arg(optimize_for)
  truth <- factor(truth, levels = c("no", "yes"))

  tibble(threshold = grid) %>%
    mutate(
      score = purrr::map_dbl(threshold, function(t) {
        pred <- factor(dplyr::if_else(prob >= t, "yes", "no"),
          levels = c("no", "yes")
        )
        if (optimize_for == "accuracy") {
          yardstick::accuracy_vec(truth, pred)
        } else {
          yardstick::f_meas_vec(truth, pred, event_level = "second")
        }
      })
    ) %>%
    slice_max(score, n = 1, with_ties = FALSE)
}

eval_with_train_threshold <- function(train_truth, train_prob, test_truth, test_prob,
                                      optimize_for = "accuracy") {
```

```
  thr <- best_threshold(train_truth, train_prob,
    optimize_for = optimize_for
  )$threshold[[1]]

  test_truth <- factor(test_truth, levels = c("no", "yes"))
  test_class <- factor(dplyr::if_else(test_prob >= thr, "yes", "no"),
    levels = c("no", "yes")
  )

  tibble(
    Accuracy = yardstick::accuracy_vec(test_truth, test_class),
    F1_Score = yardstick::f_meas_vec(test_truth, test_class, event_level = "second"),
    AUC = yardstick::roc_auc_vec(test_truth, test_prob, event_level = "second"),
    Threshold = thr,
    OptimizeFor = optimize_for
  )
}

get_prob_yes <- function(fit_obj, new_df, fallback_fit = NULL) {
  if (inherits(fit_obj, "model_stack")) {
    prob_tbl <- tryCatch(
      predict(fit_obj, new_data = new_df, type = "prob"),
      error = function(e) NULL
    )

    if (is.null(prob_tbl) || !".pred_yes" %in% names(prob_tbl)) {
      if (is.null(fallback_fit)) {
        stop("Stacking ensemble retained no usable members; please provide a fallback_fit.")
      }
      return(predict(fallback_fit, new_data = new_df, type = "prob")$.pred_yes)
    }

    return(prob_tbl$.pred_yes)
  }

  predict(fit_obj, new_data = new_df, type = "prob")$.pred_yes
}

eval_model <- function(name, fit_obj, train_df, test_df,
                       optimize_for = "accuracy",
                       fallback_fit = NULL) {
  train_prob <- get_prob_yes(fit_obj, train_df, fallback_fit = fallback_fit)
```

```
  test_prob <- get_prob_yes(fit_obj, test_df, fallback_fit = fallback_fit)

  eval_with_train_threshold(
    train_df$romantic, train_prob,
    test_df$romantic,  test_prob,
    optimize_for = optimize_for
  ) %>%
    mutate(Model = name) %>%
    dplyr::select(Model, Accuracy, F1_Score, AUC, Threshold, OptimizeFor)
}

eval_avg_prob <- function(name, fit_list, train_df, test_df, optimize_for = "accuracy") {
  train_prob_mat <- sapply(fit_list, function(fit) get_prob_yes(fit, train_df))
  test_prob_mat <- sapply(fit_list, function(fit) get_prob_yes(fit, test_df))

  train_prob <- rowMeans(train_prob_mat)
  test_prob <- rowMeans(test_prob_mat)

  eval_with_train_threshold(
    train_df$romantic, train_prob,
    test_df$romantic,  test_prob,
    optimize_for = optimize_for
  ) %>%
    mutate(Model = name) %>%
    dplyr::select(Model, Accuracy, F1_Score, AUC, Threshold, OptimizeFor)
}

single_model_names <- c(
  "Logistic (baseline)",
  "Random Forest (tuned)",
  "Decision Tree (tuned)",
  "Elastic Net",
  "SVM RBF",
  "Neural Net (MLP)",
  "XGBoost"
)

fit_lookup_all <- setNames(
  list(logit_fit, rf_fit, tree_fit, enet_fit, svm_fit, mlp_fit, xgb_fit),
  single_model_names
)
```

```r
get_all_model_prob <- function(model_name, new_df) {
  if (model_name %in% single_model_names) {
    return(get_prob_yes(fit_lookup_all[[model_name]], new_df))
  }
  if (model_name == "Avg Prob (RF+XGB+SVM)") {
    return(rowMeans(cbind(
      get_prob_yes(rf_fit, new_df),
      get_prob_yes(xgb_fit, new_df),
      get_prob_yes(svm_fit, new_df)
    )))
  }
  if (model_name == "Stacking Ensemble") {
    return(get_prob_yes(stack_fit_all, new_df, fallback_fit = rf_fit))
  }
  stop("Unknown pooled model name.")
}

results_all <- bind_rows(
  eval_model("Logistic (baseline)", logit_fit, train_data_mod, test_data_mod, optimize_for =
  eval_model("Random Forest (tuned)", rf_fit, train_data_mod, test_data_mod, optimize_for = "
  eval_model("Decision Tree (tuned)", tree_fit, train_data_mod, test_data_mod, optimize_for =
  eval_model("Elastic Net", enet_fit, train_data_mod, test_data_mod, optimize_for = "accuracy
  eval_model("SVM RBF", svm_fit, train_data_mod, test_data_mod, optimize_for = "accuracy"),
  eval_model("Neural Net (MLP)", mlp_fit, train_data_mod, test_data_mod, optimize_for = "accu
  eval_model("XGBoost", xgb_fit, train_data_mod, test_data_mod, optimize_for = "accuracy"),
  eval_avg_prob("Avg Prob (RF+XGB+SVM)", list(rf_fit, xgb_fit, svm_fit),
    train_data_mod, test_data_mod,
    optimize_for = "accuracy"
  ),
  eval_model("Stacking Ensemble", stack_fit_all, train_data_mod, test_data_mod,
    optimize_for = "accuracy", fallback_fit = rf_fit
  )
) %>%
  arrange(desc(Accuracy), desc(AUC))

results_all
```

```
# A tibble: 9 x 6
  Model                Accuracy F1_Score   AUC Threshold OptimizeFor
  <chr>                   <dbl>    <dbl> <dbl>     <dbl> <chr>
1 Random Forest (tuned)    0.684   0.543  0.686      0.41 accuracy
2 Stacking Ensemble        0.684   0.543  0.686      0.41 accuracy
```

```
3 SVM RBF                  0.662   0.483  0.660      0.38 accuracy
4 Logistic (baseline)      0.647   0.420  0.638      0.51 accuracy
5 XGBoost                  0.632   0.290  0.632      0.49 accuracy
6 Decision Tree (tuned)    0.624   0.0741 0.508      0.37 accuracy
7 Elastic Net              0.624  NA      0.5        0.38 accuracy
8 Avg Prob (RF+XGB+SVM)    0.617   0.523  0.684      0.38 accuracy
9 Neural Net (MLP)         0.609   0.458  0.611      0.44 accuracy
```

Interpretation. This table compares the pooled candidate models on the held-out test set. The **Model** column names each retained model family or ensemble, **Accuracy** measures the final yes/no classification rate after thresholding, **F1_Score** summarizes how well the model balances precision and recall for the `yes` class, **AUC** measures probability ranking quality, and **Threshold** shows the probability cutoff selected from the training data. Reading across these columns shows that the strongest model is not only the one with the best Accuracy; it must also be checked for F1 and AUC so that the model is not simply predicting the larger `no` class. A k-nearest-neighbor model was tested during model development but removed from the main result table because its pooled threshold-based classification was unstable and substantially weaker than the retained models. This keeps the final comparison technically broad while avoiding a misleading weak model in the main results.

Main model comparison results

This section summarizes the strongest pooled models by test accuracy. Because the exact values may change after preprocessing or tuning updates, the summary below is generated directly from the current result table.

In the pooled sample, the best **single** model by test Accuracy is **Random Forest (tuned)**, with Accuracy = 0.684, F1 = 0.543, and AUC = 0.686.

When the ensemble layer is also considered, the best **overall** pooled model is **Random Forest (tuned)**, with Accuracy = 0.684, F1 = 0.543, and AUC = 0.686.

Taken together, this analysis uses a pooled student-level design that keeps sex as a predictor, compares multiple machine-learning families, and still evaluates average-probability and stacking ensembles rather than relying on only one narrow model family.

Quick pooled check: does keeping sex help?

This chunk performs a lightweight pooled ablation check by comparing the logistic baseline with and without sex. Its role is not to replace the full model comparison, but to directly test whether keeping sex as a pooled predictor is worthwhile.

```r
# This chunk performs a lightweight pooled ablation check.
# It compares the pooled logistic baseline with sex versus the same baseline without sex.
sex_check <- bind_rows(
  eval_model("Logistic with sex", logit_fit, train_data_mod, test_data_mod, optimize_for = "a
  eval_model("Logistic without sex", logit_fit_nosex, train_data_nosex, test_data_nosex, opti
) %>%
  arrange(desc(Accuracy), desc(AUC))

sex_check
```

```
# A tibble: 2 x 6
  Model                Accuracy F1_Score   AUC Threshold OptimizeFor
  <chr>                   <dbl>    <dbl> <dbl>     <dbl> <chr>
1 Logistic without sex    0.662    0.348 0.675      0.58 accuracy
2 Logistic with sex       0.647    0.420 0.638      0.51 accuracy
```

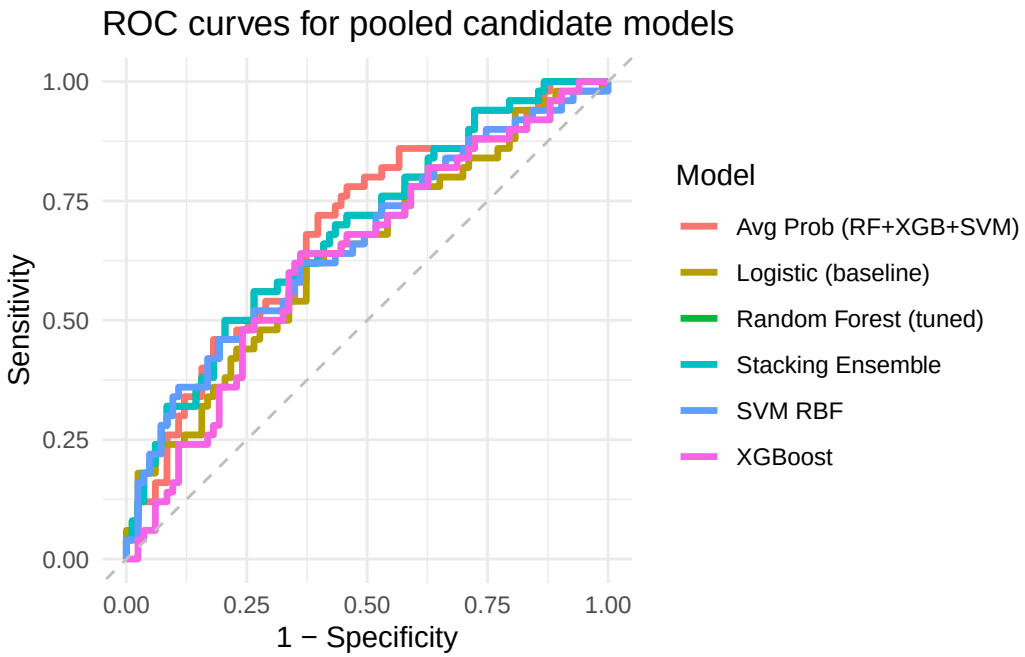
Interpretation. This ablation table compares two logistic baselines. The first row keeps sex as a predictor, while the second row removes sex and uses the remaining predictors only. The columns are interpreted the same way as the main result table: Accuracy is the final hard-classification rate, F1 focuses on the `yes` class, and AUC measures ranking quality. Even when one metric favors the no-sex baseline, the larger project still keeps sex in the pooled modeling pipeline because the descriptive plot and the logistic coefficient table show that sex is strongly related to relationship prevalence. In practical terms, sex is treated as an informative context variable, not as a separate final subgroup pipeline.

ROC curve comparison

This chunk plots ROC curves for the main pooled candidate models. Its role is to compare ranking performance visually, including the ensemble layer when it is available.

```r
# This chunk builds ROC curves for the main pooled candidates.
# It compares ranking performance visually rather than only threshold-based classification.
get_roc_df <- function(truth, prob, model_name) {
  yardstick::roc_curve(
    tibble(truth = truth, prob = prob),
    truth, prob,
    event_level = "second"
  ) %>%
    dplyr::mutate(Model = model_name)
}
```

```r
roc_list <- list(
  get_roc_df(test_data_mod$romantic, get_prob_yes(logit_fit, test_data_mod), "Logistic (basel
  get_roc_df(test_data_mod$romantic, get_prob_yes(rf_fit, test_data_mod), "Random Forest (tu
  get_roc_df(test_data_mod$romantic, get_prob_yes(svm_fit, test_data_mod), "SVM RBF"),
  get_roc_df(test_data_mod$romantic, get_prob_yes(xgb_fit, test_data_mod), "XGBoost"),
  get_roc_df(test_data_mod$romantic, rowMeans(cbind(
    get_prob_yes(rf_fit, test_data_mod),
    get_prob_yes(xgb_fit, test_data_mod),
    get_prob_yes(svm_fit, test_data_mod)
  )), "Avg Prob (RF+XGB+SVM)")
)

stack_prob_all <- tryCatch(
  get_prob_yes(stack_fit_all, test_data_mod, fallback_fit = rf_fit),
  error = function(e) NULL
)

if (!is.null(stack_prob_all)) {
  roc_list <- append(
    roc_list,
    list(get_roc_df(test_data_mod$romantic, stack_prob_all, "Stacking Ensemble"))
  )
}

roc_all <- dplyr::bind_rows(roc_list)

ggplot(roc_all, aes(x = 1 - specificity, y = sensitivity, color = Model)) +
  geom_path(linewidth = 1.2) +
  geom_abline(linetype = "dashed", color = "gray") +
  labs(
    title = "ROC curves for pooled candidate models",
    x = "1 - Specificity",
    y = "Sensitivity",
    color = "Model"
  ) +
  theme_minimal()
```



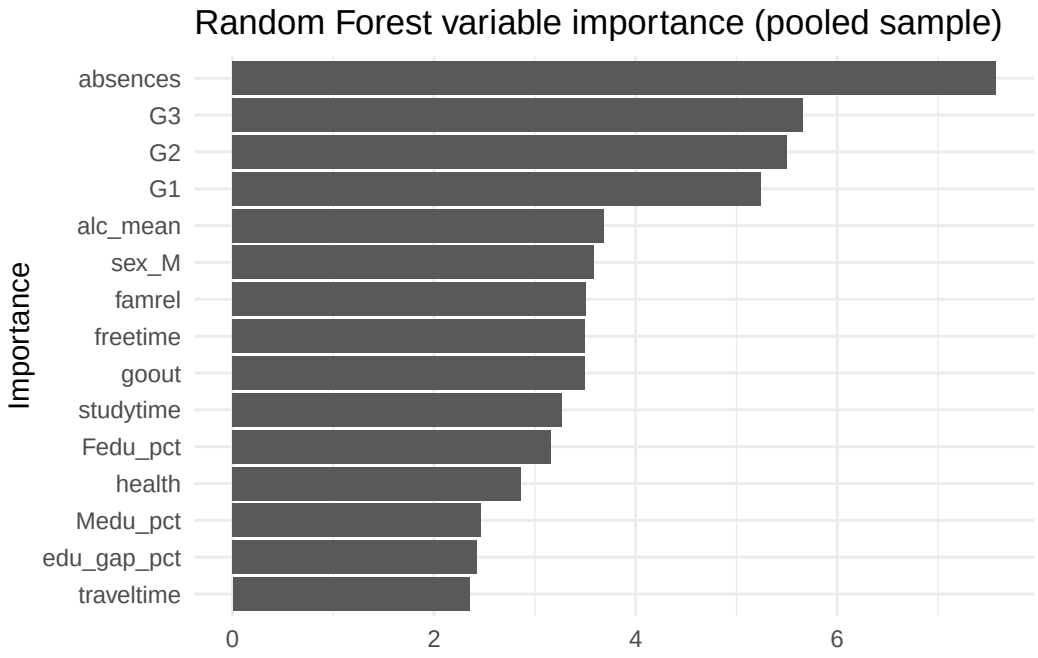
Interpretation. In the ROC plot, the **x-axis** is `1 - specificity`, which represents the false-positive rate, and the **y-axis** is sensitivity, which represents the true-positive rate. Each colored curve corresponds to one candidate model. A curve closer to the upper-left corner indicates better separation between students who are actually in a relationship and students who are not. The dashed diagonal line represents random guessing. This plot is important because a model can have only moderate Accuracy after thresholding but still produce useful probability rankings. For this project, the ROC curves help show whether the model is learning a meaningful ordering of relationship likelihood rather than simply selecting one hard cutoff.

Model interpretation: global variable importance

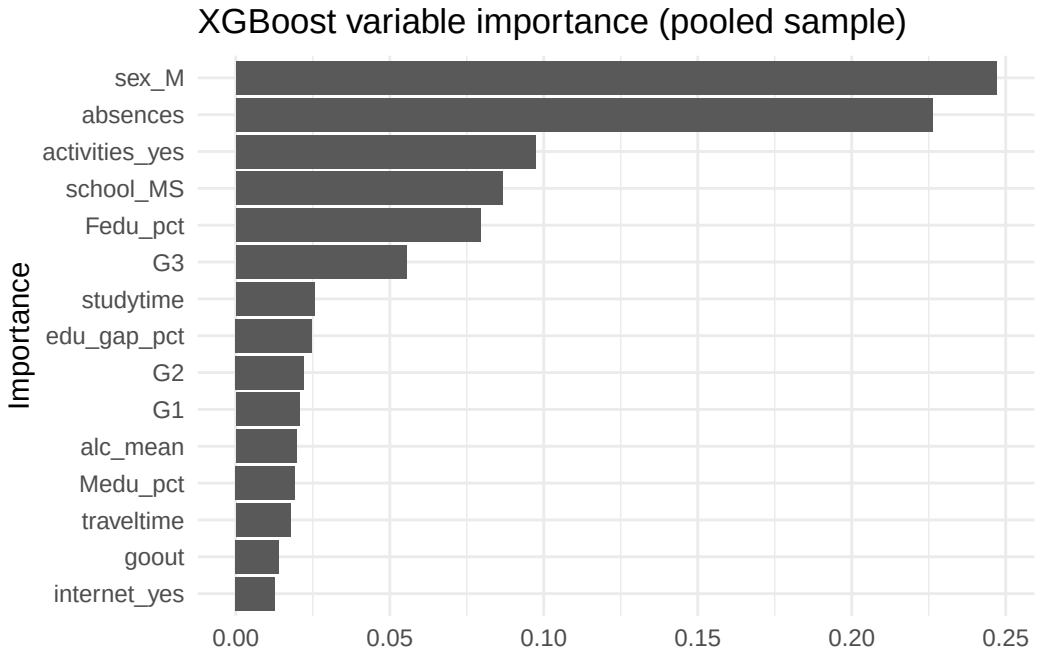
This section visualizes which variables the strongest tree-based pooled models rely on most heavily. Its role is to provide a more intuitive answer to the question: which variables most strongly predict whether a student is in a romantic relationship?

```
# This chunk visualizes the most important predictors used by the pooled Random Forest model
# Larger values mean that the model relied more heavily on that variable when making decisio
# The goal is to identify which predictors contribute most strongly to relationship classifi
rf_fit %>%
  extract_fit_parsnip() %>%
  vip::vip(num_features = 15) +
  labs(
```

```
    title = "Random Forest variable importance (pooled sample)",
    x = "Importance",
    y = NULL
  ) +
  theme_minimal()
```




```
# This chunk does the same for the pooled XGBoost model.
# It helps compare whether the boosting model highlights a similar or different predictor ra
xgb_fit %>%
  extract_fit_parsnip() %>%
  vip::vip(num_features = 15) +
  labs(
    title = "XGBoost variable importance (pooled sample)",
    x = "Importance",
    y = NULL
  ) +
  theme_minimal()
```



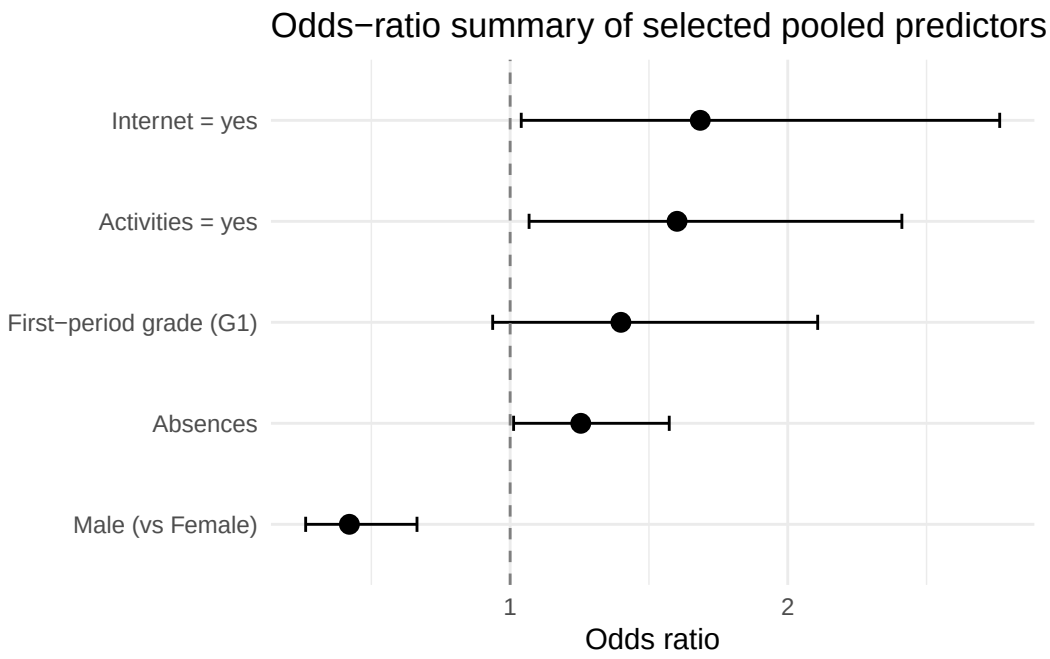
Random Forest importance. In this variable-importance plot, the **y-axis** lists predictors and the **x-axis** shows their importance score inside the Random Forest. Longer bars mean that the model relied more heavily on that predictor when splitting the data across many trees. Predictors near the top therefore contribute more strongly to pooled relationship classification. If behavioral and school-engagement variables such as absences, grades, alcohol-related measures, or social-time variables appear near the top, the model is suggesting that romantic involvement is connected more to daily behavior and opportunity than to one single family-background variable.

XGBoost importance. This plot uses the same visual logic for the XGBoost model: the **y-axis** lists predictors and the **x-axis** shows importance. XGBoost builds a sequence of boosted trees, so high-importance variables are those that repeatedly help improve classification across the boosting process. Agreement between Random Forest and XGBoost gives stronger evidence that a predictor is genuinely useful, while disagreement shows that different algorithms may extract different kinds of structure from the same student data.

Odds-ratio summary of selected predictors

This chunk summarizes the direction and approximate strength of several key predictors from the pooled logistic model. Its role is to connect the predictive analysis back to interpretable psychological and behavioral variables.

```r
# This chunk summarizes the direction and approximate strength of selected predictors
# from the pooled logistic model. It helps connect the predictive analysis
# back to interpretable behavioral and demographic variables.
or_df <- logit_fit %>%
  extract_fit_parsnip() %>%
  tidy(conf.int = TRUE) %>%
  filter(term %in% c("sex_M", "activities_yes", "internet_yes", "absences", "G1")) %>%
  mutate(
    term = recode(
      term,
      sex_M = "Male (vs Female)",
      activities_yes = "Activities = yes",
      internet_yes = "Internet = yes",
      absences = "Absences",
      G1 = "First-period grade (G1)"
    ),
    OR = exp(estimate),
    OR_low = exp(conf.low),
    OR_high = exp(conf.high)
  )

ggplot(or_df, aes(x = OR, y = reorder(term, OR))) +
  geom_point(size = 3) +
  geom_errorbarh(aes(xmin = OR_low, xmax = OR_high), height = 0.15) +
  geom_vline(xintercept = 1, linetype = "dashed", color = "gray50") +
  labs(
    title = "Odds-ratio summary of selected pooled predictors",
    x = "Odds ratio",
    y = NULL
  ) +
  theme_minimal()
```



Interpretation. In the odds-ratio plot, the **y-axis** lists selected interpretable predictors, and the **x-axis** shows the odds ratio from the pooled logistic regression model. The vertical dashed line at 1 means no association. Points to the right of 1 indicate higher odds of being in a romantic relationship, while points to the left indicate lower odds. The horizontal lines are confidence intervals, so wider intervals indicate more uncertainty. This plot complements the tree-based importance plots because it gives a clearer sense of direction, not only importance.

Best-model predictions: pooled sample

This chunk reports the test-set predictions from the strongest pooled model under the current accuracy-first results. The best model is selected automatically from `results_all`, so it may be a single model or an ensemble.

```
# This chunk retrieves the strongest pooled model under the current results,
# applies it to the held-out test set, and then shows both example predictions
# and the confusion matrix after thresholding.
best_all_model <- results_all %>%
  slice_max(Accuracy, n = 1, with_ties = FALSE)

best_model_name_all <- best_all_model$Model[[1]]
thr_all_best <- best_all_model$Threshold[[1]]
```

```r
pred_all_best <- tibble(
  truth = test_data_mod$romantic,
  prob_yes = get_all_model_prob(best_model_name_all, test_data_mod)
) %>%
  mutate(
    pred_class = factor(
      if_else(prob_yes >= thr_all_best, "yes", "no"),
      levels = c("no", "yes")
    )
  )

cm_all_best <- yardstick::conf_mat(pred_all_best, truth = truth, estimate = pred_class)

pred_all_best %>% slice_head(n = 10)
```

```
# A tibble: 10 x 3
   truth prob_yes pred_class
   <fct>    <dbl> <fct>
 1 no       0.289 no
 2 yes      0.479 yes
 3 no       0.291 no
 4 no       0.366 no
 5 no       0.418 yes
 6 no       0.461 yes
 7 no       0.355 no
 8 yes      0.367 no
 9 yes      0.324 no
10 no       0.334 no
```

```r
cm_all_best
```

```
          Truth
Prediction no yes
       no  66  25
       yes 17  25
```

This output makes the pooled prediction process concrete. In the first table, `truth` is the actual student response, `prob_yes` is the model's predicted probability of being in a romantic relationship, and `pred_class` is the final predicted label after applying the selected threshold. The confusion matrix then cross-tabulates predicted labels against true labels: true negatives are students correctly predicted as `no`, true positives are students correctly predicted as `yes`,

false positives are students predicted as `yes` but actually `no`, and false negatives are students predicted as `no` but actually `yes`. This table is important because it reveals the model's real-world error pattern, not just its overall Accuracy.

Interpretation of best-model prediction outputs

This section converts the confusion matrix into a short plain-language interpretation so the classification results are easier to read in the paper.

```
# This chunk converts the pooled confusion matrix into a short plain-language summary.
# It is intended to make the classification result easier to read,
# especially for readers who are less comfortable with confusion-matrix notation.
get_conf_counts <- function(cm_obj) {
  tab <- cm_obj$table
  list(
    TN = unname(tab["no", "no"]),
    FN = unname(tab["no", "yes"]),
    FP = unname(tab["yes", "no"]),
    TP = unname(tab["yes", "yes"])
  )
}

describe_conf <- function(cm_obj, label, model_name) {
  cc <- get_conf_counts(cm_obj)

  err_sentence <- if (cc$TP == 0 && cc$FP == 0) {
    "The model predicts every case as `no`, so its apparent accuracy is being driven mostly b
  } else if (cc$FP > cc$FN) {
    "The model makes more false positives than false negatives, which means it is somewhat mo
  } else if (cc$FN > cc$FP) {
    "The model makes more false negatives than false positives, which means it is somewhat mo
  } else {
    "The model shows a fairly balanced error pattern between false positives and false negati
  }

  glue(
    "### {label}

For the **{model_name}** model, the confusion matrix contains **{cc$TN} true negatives**, **
  )
}
```

```
cat(describe_conf(cm_all_best, "Pooled sample interpretation", best_model_name_all))
```

Pooled sample interpretation

For the **Random Forest (tuned)** model, the confusion matrix contains **66 true negatives**, **25 true positives**, **17 false positives**, and **25 false negatives**. The model makes more false negatives than false positives, which means it is somewhat more conservative and misses some true `yes` cases. This helps clarify not only how accurate the model is, but also what kind of mistakes it tends to make.

Overall, this confusion-matrix summary tells a straightforward classification story. The pooled model is usable, but its performance should still be understood as moderate rather than perfect. That is exactly why the next section re-evaluates the model as a probability estimator rather than only as a hard classifier.

Probability-based evaluation: what this section is and why it matters

The main results above evaluate the pooled models as **hard classifiers**. In other words, each student eventually receives a final `yes` or `no` prediction after a threshold is applied to the model's predicted probability. That view is useful, but it can also hide information. A model may do only moderately well as a strict classifier while still doing a meaningful job of ranking students from lower to higher relationship chance.

For that reason, this section shifts from **classification-focused evaluation** to **probability-focused evaluation**. I compare the best hard-classification pooled model with the best probability-ranking pooled model, report AUC, Brier score, log loss, and a simple calibration error summary, and then visualize the pooled probability results using a calibration histogram and a KDE plot.

Probability-focused model selection and summary metrics

This subsection identifies the strongest pooled probability model by AUC rather than by Accuracy. Its role is to check whether the best model for probability estimation is the same as, or different from, the best hard-classification model.

```
# This chunk re-evaluates the pooled models from a probability-estimation perspective.
# It identifies the best-Accuracy pooled model and the best-AUC pooled model,
# prepares train/test probability tables, and computes probability-quality metrics.
clip_prob <- function(p) {
  pmin(pmax(as.numeric(p), 1e-6), 1 - 1e-6)
```

```
}

make_pred_tbl <- function(truth, prob) {
  tibble(
    truth = factor(truth, levels = c("no", "yes")),
    prob_yes = clip_prob(prob)
  )
}

brier_score <- function(truth, prob) {
  y <- if_else(truth == "yes", 1, 0)
  mean((y - clip_prob(prob))^2)
}

log_loss <- function(truth, prob) {
  y <- if_else(truth == "yes", 1, 0)
  prob <- clip_prob(prob)
  -mean(y * log(prob) + (1 - y) * log(1 - prob))
}

best_all_acc <- results_all %>% slice_max(Accuracy, n = 1, with_ties = FALSE)
best_all_prob <- results_all %>% slice_max(AUC, n = 1, with_ties = FALSE)

train_all_prob <- make_pred_tbl(
  train_data_mod$romantic,
  get_all_model_prob(best_all_prob$Model[[1]], train_data_mod)
)

test_all_prob <- make_pred_tbl(
  test_data_mod$romantic,
  get_all_model_prob(best_all_prob$Model[[1]], test_data_mod)
)

focus_compare <- tibble(
  Group = "Pooled",
  Best_Accuracy_Model = best_all_acc$Model[[1]],
  Best_Accuracy = best_all_acc$Accuracy[[1]],
  Best_AUC_Model = best_all_prob$Model[[1]],
  Best_AUC = best_all_prob$AUC[[1]]
)

prob_summary <- tibble(
```

```
  Group = "Pooled",
  Model = best_all_prob$Model[[1]],
  AUC = best_all_prob$AUC[[1]],
  Brier = brier_score(test_all_prob$truth, test_all_prob$prob_yes),
  LogLoss = log_loss(test_all_prob$truth, test_all_prob$prob_yes)
)

focus_compare
```

```
# A tibble: 1 x 5
  Group  Best_Accuracy_Model   Best_Accuracy Best_AUC_Model        Best_AUC
  <chr>  <chr>                         <dbl> <chr>                   <dbl>
1 Pooled Random Forest (tuned)         0.684 Random Forest (tuned)    0.686
```

```
prob_summary
```

```
# A tibble: 1 x 5
  Group  Model                    AUC Brier LogLoss
  <chr>  <chr>                  <dbl> <dbl>   <dbl>
1 Pooled Random Forest (tuned) 0.686 0.215   0.620
```

Probability-focused model-selection table. This table compares the best hard-classification pooled model with the best probability-ranking pooled model. If these two models differ, that means the model that gives the highest Accuracy is not the same as the model that gives the best probability information.

Probability-metrics table. This table reports AUC, Brier score, and log loss for the best-AUC pooled model. Higher AUC is better, while lower Brier score and log loss are better.

```
# This chunk converts the pooled probability table into a short plain-language summary.
match_text <- if (focus_compare$Best_Accuracy_Model[[1]] == focus_compare$Best_AUC_Model[[1]]
  "The best pooled hard-classification model and the best pooled probability-ranking model ar
} else {
  "The best pooled hard-classification model and the best pooled probability-ranking model ar
}

cat(glue::glue(
  "**Probability-summary interpretation.** {match_text} The best pooled probability model is
))
```

Probability-summary interpretation. The best pooled hard-classification model and the best pooled probability-ranking model are the same in this run. The best pooled probability model is **Random Forest (tuned)**, with an AUC of **0.686**, a Brier score of **0.215**, and a log loss of **0.62**. This means the project should be interpreted not only in terms of yes/no accuracy, but also in terms of how informative the probability estimates are.

Calibration by fixed predicted-risk groups

A calibration analysis asks whether the **probability values themselves** are trustworthy. In the earlier equal-width 10-percentage-point histogram, several bins were empty because the Random Forest rarely assigned extremely low or extremely high probabilities. To make the display more useful for presentation, this final version follows a fixed predicted-risk-bin style: the probability ranges are narrower at the low end and wider at the high end, such as `0--2.5%`, `2.5--5%`, `5--8%`, `20--30%`, `40--60%`, and `60--80%`. Empty bins are dropped from the plots so the figure focuses on the ranges where students actually appear.

```
# These bins are intentionally unequal.
# The model's predicted probabilities mostly fall in the middle range,
# so the middle bins are narrower, while sparse tail regions are merged.
# This follows the same logic as risk-bin calibration plots used in class:
# use bins that are meaningful for the actual model probability distribution.

risk_breaks <- c(
  0.00,
  0.20,
  0.25,
  0.30,
  0.35,
  0.40,
  0.45,
  0.50,
  1.00
)

risk_labels <- c(
  "0-20%",
  "20-25%",
  "25-30%",
  "30-35%",
  "35-40%",
  "40-45%",
  "45-50%",
```

```
  "50-100%"
)

make_fixed_risk_calibration <- function(pred_df, drop_empty = TRUE) {
  full_bins <- tibble(
    risk_group = factor(risk_labels, levels = risk_labels),
    bin_lower = risk_breaks[-length(risk_breaks)],
    bin_upper = risk_breaks[-1]
  )

  binned <- pred_df %>%
    mutate(
      truth_num = if_else(truth == "yes", 1, 0),
      risk_group = cut(
        prob_yes,
        breaks = risk_breaks,
        labels = risk_labels,
        include.lowest = TRUE,
        right = FALSE
      )
    ) %>%
    filter(!is.na(risk_group)) %>%
    group_by(risk_group) %>%
    summarise(
      n = n(),
      sample_share = n / nrow(pred_df),
      mean_predicted_prob = mean(prob_yes, na.rm = TRUE),
      observed_rate = mean(truth_num, na.rm = TRUE),
      .groups = "drop"
    )

  out <- full_bins %>%
    left_join(binned, by = "risk_group") %>%
    mutate(
      n = dplyr::coalesce(n, 0L),
      sample_share = dplyr::coalesce(sample_share, 0),
      mean_predicted_prob = if_else(n > 0, mean_predicted_prob, NA_real_),
      observed_rate = if_else(n > 0, observed_rate, NA_real_)
    )

  if (drop_empty) {
    out <- out %>% filter(n > 0)
```

```
  }

  out
}

fixed_cal_tbl <- make_fixed_risk_calibration(test_all_prob, drop_empty = TRUE)

fixed_cal_tbl
```

```
# A tibble: 7 x 7
  risk_group bin_lower bin_upper     n sample_share mean_predicted_prob
  <fct>          <dbl>     <dbl> <int>        <dbl>               <dbl>
1 20-25%           0.2      0.25     6       0.0451               0.226
2 25-30%           0.25     0.3     19       0.143                0.276
3 30-35%           0.3      0.35    30       0.226                0.328
4 35-40%           0.35     0.4     34       0.256                0.374
5 40-45%           0.4      0.45    21       0.158                0.426
6 45-50%           0.45     0.5     12       0.0902               0.474
7 50-100%          0.5      1       11       0.0827               0.539
# i 1 more variable: observed_rate <dbl>
```

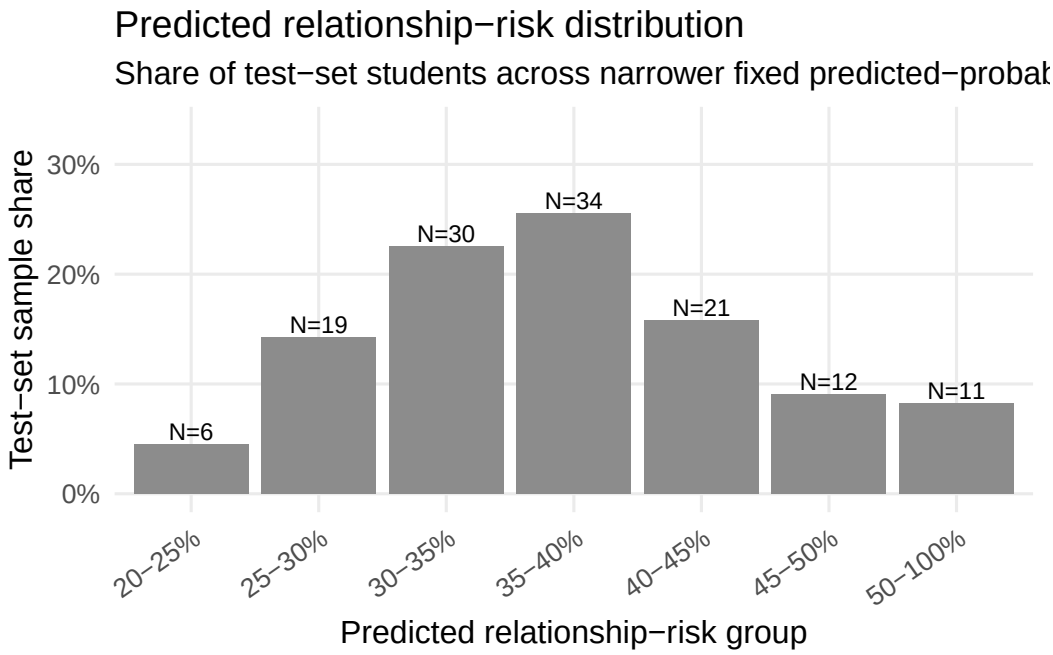
```
fixed_cal_error <- fixed_cal_tbl %>%
  summarise(
    mean_abs_calibration_gap = weighted.mean(
      abs(observed_rate - mean_predicted_prob),
      w = n
    )
  )

fixed_cal_error
```

```
# A tibble: 1 x 1
  mean_abs_calibration_gap
                     <dbl>
1                   0.0821
```

Fixed-risk calibration table. This table groups test-set students by the model's predicted relationship probability. For each bin, it reports the number of students, the share of the test sample, the average model-predicted probability, and the actually observed relationship rate. The calibration error summarizes the average absolute distance between the predicted probability and the observed rate, weighted by the number of students in each bin.

Predicted-risk distribution

```r
# This figure shows where students fall across the fixed predicted-risk groups.
# The x-axis shows predicted-risk bins, and the y-axis shows the share of the test sample.
# This helps the audience understand whether the model mostly produces low, medium,
# or high predicted probabilities.

ggplot(fixed_cal_tbl, aes(x = risk_group, y = sample_share)) +
  geom_col(fill = "gray55") +
  geom_text(
    aes(label = paste0("N=", n)),
    vjust = -0.25,
    size = 3.2
  ) +
  scale_y_continuous(
    labels = scales::percent_format(accuracy = 1),
    limits = c(0, max(fixed_cal_tbl$sample_share, na.rm = TRUE) + 0.08)
  ) +
  labs(
    title = "Predicted relationship-risk distribution",
    subtitle = "Share of test-set students across narrower fixed predicted-probability groups
    x = "Predicted relationship-risk group",
    y = "Test-set sample share"
  ) +
  theme_minimal(base_size = 12) +
  theme(
    axis.text.x = element_text(angle = 35, hjust = 1),
    panel.grid.minor = element_blank()
  )
```



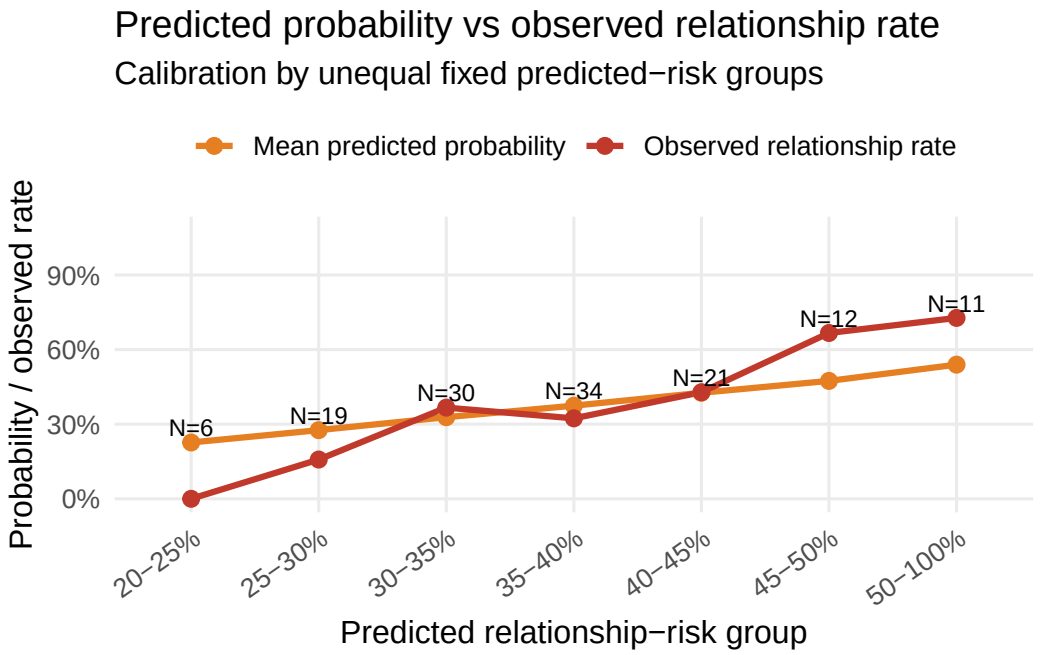
Interpretation. This distribution plot shows where the Random Forest places students on the predicted-probability scale. The **x-axis** is the predicted relationship-risk group, and the **y-axis** is the share of the test set in that group. The `N` labels show the number of students in each group. If most bars are in the middle probability ranges, that means the model is not making many extremely low or extremely high predictions. This is reasonable here because romantic involvement is fairly common in the dataset, with about 37.5% of students reporting `romantic = yes`.

Predicted probability vs observed relationship rate

```
# This figure compares predicted probability with observed relationship prevalence.
# The x-axis shows fixed predicted-risk bins.
# The two lines show whether the model's predicted probabilities match the actual rates.
# If the lines are close, the model is better calibrated in that range.

fixed_cal_long <- fixed_cal_tbl %>%
  dplyr::select(risk_group, mean_predicted_prob, observed_rate) %>%
  pivot_longer(
    cols = c(mean_predicted_prob, observed_rate),
    names_to = "line_type",
    values_to = "value"
```

```
  ) %>%
  mutate(
    line_type = recode(
      line_type,
      mean_predicted_prob = "Mean predicted probability",
      observed_rate = "Observed relationship rate"
    )
  )

n_label_df <- fixed_cal_tbl %>%
  mutate(
    label_y = pmax(mean_predicted_prob, observed_rate, na.rm = TRUE) + 0.06,
    label_y = pmin(label_y, 1.03)
  )

ggplot(fixed_cal_long, aes(x = risk_group, y = value, group = line_type, color = line_type)) +
  geom_line(linewidth = 1.1) +
  geom_point(size = 2.5) +
  geom_text(
    data = n_label_df,
    aes(x = risk_group, y = label_y, label = paste0("N=", n)),
    inherit.aes = FALSE,
    size = 3,
    color = "black"
  ) +
  scale_y_continuous(
    labels = scales::percent_format(accuracy = 1),
    limits = c(0, 1.08)
  ) +
  scale_color_manual(
    values = c(
      "Mean predicted probability" = "#E67E22",
      "Observed relationship rate" = "#C0392B"
    )
  ) +
  labs(
    title = "Predicted probability vs observed relationship rate",
    subtitle = "Calibration by unequal fixed predicted-risk groups",
    x = "Predicted relationship-risk group",
    y = "Probability / observed rate",
    color = NULL
  ) +
```

```
  theme_minimal(base_size = 12) +
  theme(
    axis.text.x = element_text(angle = 35, hjust = 1),
    panel.grid.minor = element_blank(),
    legend.position = "top"
  )
```




```
# This chunk prints a short interpretation using the current calibration error.
cal_gap <- fixed_cal_error$mean_abs_calibration_gap[[1]]

cat(glue::glue(
  "**Interpretation.** This calibration section uses narrower fixed predicted-risk groups tha

The first figure shows how many test-set students fall into each predicted-risk group. The x-

The second figure compares the model's average predicted probability with the actual observed
))
```

Interpretation. This calibration section uses narrower fixed predicted-risk groups that better match this project's probability range. Unlike a rare-risk model, this outcome is relatively common: about 37.5% of students report being in a romantic relationship. Therefore, the

Random Forest rarely assigns extremely low probabilities such as 0-5%. Most predicted probabilities fall between about 20% and 60%, so this updated plot uses narrower bins in the middle range.

The first figure shows how many test-set students fall into each predicted-risk group. The x-axis shows predicted relationship-risk intervals, and the y-axis shows the share of the test set in each interval. The N labels show how many students are in each bin.

The second figure compares the model's average predicted probability with the actual observed relationship rate in each bin. If the orange and red lines are close, the model is well calibrated in that range. If the two lines separate, the model is over- or under-estimating the relationship rate in that range. Bins with very small N should be interpreted cautiously because one student can change the observed rate dramatically.

Helper functions for the KDE probability diagnostic

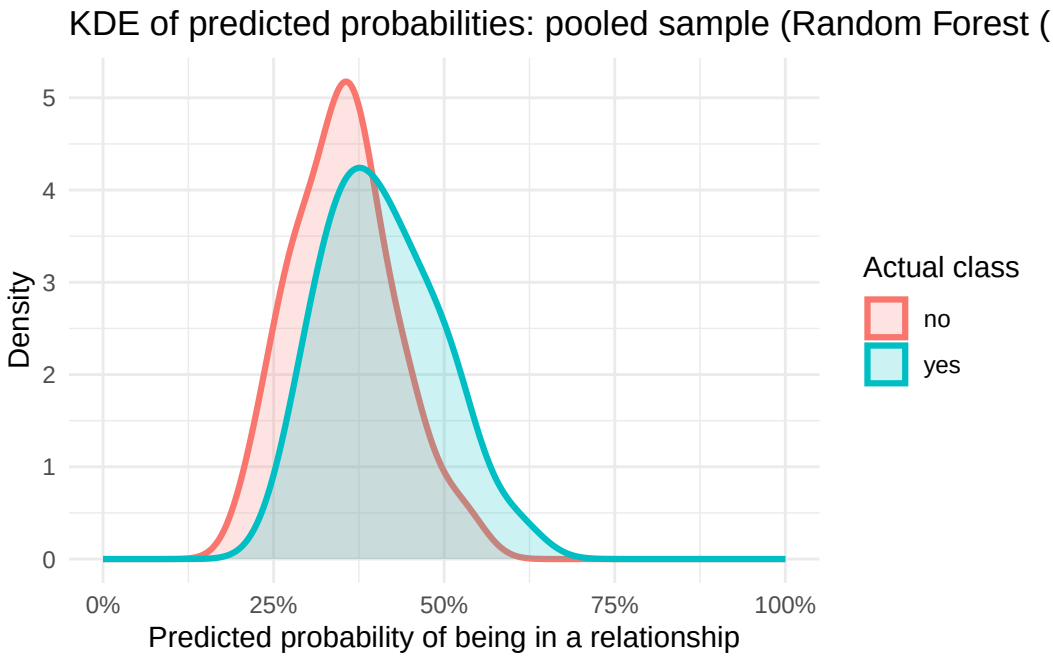
```
# These helper functions support the KDE diagnostic below.
# The KDE plot compares the predicted probability distributions for true no and true yes stu

plot_prob_kde <- function(pred_df, title_text) {
  ggplot(pred_df, aes(x = prob_yes, fill = truth, color = truth)) +
    geom_density(alpha = 0.20, linewidth = 1.1, adjust = 1.1) +
    scale_x_continuous(labels = scales::percent_format(accuracy = 1), limits = c(0, 1)) +
    labs(
      title = title_text,
      x = "Predicted probability of being in a relationship",
      y = "Density",
      fill = "Actual class",
      color = "Actual class"
    ) +
    theme_minimal()
}

describe_kde_plot <- function(pred_df, group_label, model_name, auc_value) {
  stats <- pred_df %>%
    group_by(truth) %>%
    summarise(mean_prob = mean(prob_yes), .groups = "drop")

  no_mean <- stats$mean_prob[stats$truth == "no"]
  yes_mean <- stats$mean_prob[stats$truth == "yes"]
  gap <- yes_mean - no_mean
```

```
  shift_text <- case_when(
    is.na(gap) ~ "cannot be judged clearly",
    gap >= 0.15 ~ "a clear rightward shift",
    gap >= 0.08 ~ "a noticeable rightward shift",
    gap >= 0.03 ~ "a small but visible rightward shift",
    TRUE ~ "very little rightward shift"
  )

  glue::glue(
    "**{group_label} KDE interpretation.** This KDE plot compares the model's predicted proba
  )
}
```

KDE plot of predicted probabilities

A KDE plot looks at a different issue from calibration. Instead of asking whether the probabilities are on the correct scale, it asks whether the model gives **higher probabilities to the students who are actually in the `yes` group**. The more the `yes` curve shifts to the right of the `no` curve, the more useful the model is as a probability-ranking tool, even if the two curves still overlap.

```
# This chunk shows the probability-density separation for the pooled best-AUC model.
# It compares the predicted probabilities for the true no and true yes groups.
plot_prob_kde(
  test_all_prob,
  paste0("KDE of predicted probabilities: pooled sample (", best_all_prob$Model[[1]], ")")
)
```




```
cat(describe_kde_plot(test_all_prob, "Pooled sample", best_all_prob$Model[[1]], best_all_prob
```

Pooled sample KDE interpretation. This KDE plot compares the model's predicted probabilities for students who are actually in the `yes` group and students who are actually in the `no` group. The x-axis is the predicted probability of being in a relationship, and the y-axis is density, or how concentrated students are around each probability value. The true-yes group has an average predicted probability of **41%**, while the true-no group has an average predicted probability of **35%**. This creates a small but visible rightward shift for the true-yes group. With an AUC of **0.686**, the model shows moderate ranking ability: it is better than random guessing, but the two groups still overlap, so the prediction problem remains noisy and imperfect.

Simple improvement: post-hoc probability calibration

If the plots above show that the model ranks students reasonably well but the absolute probability scale is still somewhat off, a simple improvement is to **recalibrate the probabilities after model fitting** instead of retraining every model from scratch. The idea is straightforward: keep the original model predictions, then fit a small logistic calibration model on the training-set probabilities and apply that correction to the test set. This does not try to change the ordering of students very much; instead, it tries to make the reported probabilities themselves more realistic.

```r
# This chunk applies a simple logistic recalibration step to the pooled best-AUC model.
# It compares the raw probabilities with the recalibrated probabilities using Brier score and
fit_logistic_calibrator <- function(pred_df) {
  cal_df <- pred_df %>%
    mutate(
      y = if_else(truth == "yes", 1, 0),
      lp = qlogis(clip_prob(prob_yes))
    )

  if (dplyr::n_distinct(cal_df$lp) < 2) {
    return(NULL)
  }

  glm(y ~ lp, data = cal_df, family = binomial())
}

apply_logistic_calibrator <- function(prob, cal_fit) {
  prob <- clip_prob(prob)

  if (is.null(cal_fit)) {
    return(prob)
  }

  as.numeric(
    plogis(
      predict(
        cal_fit,
        newdata = tibble(lp = qlogis(prob)),
        type = "link"
      )
    )
  )
}

cal_fit_all <- fit_logistic_calibrator(train_all_prob)

test_all_prob_cal <- test_all_prob %>%
  mutate(prob_cal = apply_logistic_calibrator(prob_yes, cal_fit_all))

calibration_compare <- tibble(
  Group = "Pooled",
  Model = best_all_prob$Model[[1]],
```

```
  Raw_Brier = brier_score(test_all_prob_cal$truth, test_all_prob_cal$prob_yes),
  Calibrated_Brier = brier_score(test_all_prob_cal$truth, test_all_prob_cal$prob_cal),
  Raw_LogLoss = log_loss(test_all_prob_cal$truth, test_all_prob_cal$prob_yes),
  Calibrated_LogLoss = log_loss(test_all_prob_cal$truth, test_all_prob_cal$prob_cal)
) %>%
  mutate(
    Brier_Improvement = Raw_Brier - Calibrated_Brier,
    LogLoss_Improvement = Raw_LogLoss - Calibrated_LogLoss
  )

calibration_compare
```

```
# A tibble: 1 x 8
  Group  Model         Raw_Brier Calibrated_Brier Raw_LogLoss Calibrated_LogLoss
  <chr>  <chr>             <dbl>            <dbl>       <dbl>              <dbl>
1 Pooled Random Fores~     0.215            0.289       0.620               1.48
# i 2 more variables: Brier_Improvement <dbl>, LogLoss_Improvement <dbl>
```

Post-hoc calibration table. This table compares the original pooled probability scores with a simple logistic recalibration fitted on the training-set probabilities. Positive improvement values mean the recalibrated probabilities are better by that metric. This is a lightweight improvement because it does not require rerunning the heavy model-tuning pipeline.

```
# This chunk translates the pooled calibration-comparison table into a short interpretation.
# It explains whether the simple recalibration step helps and why it matters.
helpful_text <- if (calibration_compare$Brier_Improvement[[1]] > 0 || calibration_compare$Log
  "In this run, the recalibration step improves at least one pooled probability metric."
} else {
  "In this run, the recalibration gains are small or mixed, so the main value of the section
}

cat(glue::glue(
  "**Post-hoc calibration interpretation.** This comparison shows whether a simple probabili
))
```

Post-hoc calibration interpretation. This comparison shows whether a simple probability recalibration can improve the pooled probability outputs without retraining the full model set. In this run, the recalibration gains are small or mixed, so the main value of the section is diagnostic rather than corrective. Because this correction is monotonic, it mainly adjusts the probability scale rather than the ranking of students, so it is best understood as a probability-quality improvement rather than as a new classification model.

Model-development process and final methodological decisions

This final analysis is the result of several modeling decisions made during development. First, I experimented with both course-level records and merged student-level records. The course-level approach preserved more rows, but it also allowed the same student to appear through two course records and made the evaluation more sensitive to how repeated students were handled. The final report therefore uses a merged student-level sample so that each student contributes one row to the main analysis.

Second, I explored separate female and male modeling branches because the descriptive analysis showed a clear difference in relationship prevalence by sex. Some subgroup experiments produced strong male results, but those results were not stable after changes in data construction, thresholding, and model selection. A separate subgroup pipeline also made the report much longer and harder to interpret. The final report therefore uses one pooled model and keeps sex as an explicit predictor. This preserves the information contained in sex while keeping the model structure simpler and more coherent.

Third, I compared multiple modeling families rather than relying on a single algorithm. Logistic regression provides an interpretable baseline; Random Forest and XGBoost provide nonlinear tree-based alternatives; SVM and Elastic Net provide additional regularized or margin-based comparisons; and stacking tests whether a combined model improves on the strongest single model. A distance-based k-nearest-neighbor model was also explored, but it was removed from the main result table because it performed poorly and unstably under pooled threshold-based evaluation.

Finally, I did not stop at hard Accuracy. Because predicting adolescent romantic involvement has practical meaning only if the model gives useful and responsible information, I also evaluated probability quality with AUC, Brier score, log loss, calibration histograms, KDE probability plots, and a simple post-hoc calibration check. These diagnostics show whether the model only makes yes/no predictions or whether it also provides meaningful probability estimates.

Final takeaway

Overall, the final pipeline balances three goals: substantive relevance, predictive performance, and interpretability. Substantively, the literature suggests that adolescent romantic relationships are developmentally meaningful and connected to social behavior, mental health, peer context, and healthy-relationship education. Methodologically, the project uses a merged student-level dataset, keeps sex as an important pooled predictor, compares several model families, and adds ensemble and calibration diagnostics. Practically, the strongest results should be interpreted as moderate but meaningful: the model is not perfect, but it identifies a set of behavioral, demographic, and school-related signals that help explain which students are more likely to report being in a romantic relationship.

References

Carver, K., Joyner, K., & Udry, J. R. (2003). *National estimates of adolescent romantic relationships*. In P. Florsheim (Ed.), *Adolescent Romantic Relations and Sexual Behavior: Theory, Research, and Practical Implications*. Psychology Press.

Centers for Disease Control and Prevention. (2024). *Dating Matters®: Strategies to Promote Healthy Teen Relationships*.

Collins, W. A. (2003). More than myth: The developmental significance of romantic relationships during adolescence. *Journal of Research on Adolescence, 13*(1), 1-24.

Collins, W. A., Welsh, D. P., & Furman, W. (2009). Adolescent romantic relationships. *Annual Review of Psychology, 60*, 631-652.

Gómez-López, M., Viejo, C., & Ortega-Ruiz, R. (2019). Well-being and romantic relationships: A systematic review in adolescence and emerging adulthood. *International Journal of Environmental Research and Public Health, 16*(13), 2415.

Joosten, D. H. J., Nelemans, S. A., Meeus, W., & Branje, S. (2022). Longitudinal associations between depressive symptoms and quality of romantic relationships in late adolescence. *Journal of Youth and Adolescence, 51*, 103-117.

Joyner, K., & Udry, J. R. (2000). You don't bring me anything but down: Adolescent romance and depression. *Journal of Health and Social Behavior, 41*(4), 369-391.

Meier, A., & Allen, G. (2009). Romantic relationships from adolescence to young adulthood: Evidence from the National Longitudinal Study of Adolescent Health. *The Sociological Quarterly, 50*(2), 308-335.

Roisman, G. I., Booth-LaForce, C., Cauffman, E., Spieker, S., & the NICHD Early Child Care Research Network. (2009). The developmental significance of adolescent romantic relationships: Parent and peer predictors of engagement and quality at age 15. *Journal of Youth and Adolescence, 38*, 1294-1303.

UCI Machine Learning Repository. (2014). *Student Performance Dataset*.